SISTEMAS OPERATIVOS

UNIDAD IV

Contenidos

Definición de archivo y sus atributos. 3
Concepto de archivo. 3
Nomenclatura de archivos. 4
Atributos de archivo. 6
Más atributos de archivos. 6
Operaciones con los archivos. 7
Referencias y bibliografía de la sección. 11
Tipos de archivos. 11
Reconocimiento del tipo de archivo. 15
Tipos de archivos en UNIX. 16
Referencias y bibliografía de la sección. 17
Tipos de acceso. 18
Acceso secuencial. 18
Acceso aleatorio o directo. 19
Referencias y bibliografía de la sección. 19
Estructura de un archivo. 19
Referencias y bibliografía de la sección. 23
Concepto de partición y su formato. 23
Estructuras de almacenamiento. 23
Particiones y montaje. 28
Estructura del volumen en UNIX. 30
Referencias y bibliografía de la sección. 30
Esquemas de asignación de bloques. 30
Asignación contigua. 31
Asignación de lista enlazada (ligada). 34
Asignación de lista enlazada utilizando una tabla en memoria (FAT). 35
Asignación indexada (nodos-i). 37
Nodos-i en UNIX. 39
Asignación en UNIX. 40
Referencias y bibliografía de la sección. 42

Organización de bloques y espacios libres. 42
Tamaño de bloque. 43
Registro de bloques libres. 44
Almacenamiento de la lista de bloques libres en una lista enlazada. 44
Mapa de bits. 45
Cuotas de disco. 47
Agrupamiento. 49
Recuento. 49
Referencias y bibliografía de la sección. 49
Directorios. 49
Introducción a los directorios. 50
Sistemas de directorios de un solo nivel. 50
Sistemas de directorios jerárquicos. 51
Directorios en UNIX. 52
Nombres de rutas. 53
Operaciones de directorios. 56
Referencias y bibliografía de la sección. 57
Montaje. 57
Referencias y bibliografía de la sección. 60
La bitácora. 60
Referencias y bibliografía de la sección. 64
Sistemas compartidos. 64
Derechos de acceso. 64
Acceso simultaneo. 66
Archivos compartidos. 66
Referencias y bibliografía de la sección. 69
El sistema de archivos y su estructura. 69
Referencias y bibliografía de la sección. 72
Sistemas de archivos en Linux y Windows. 72
El sistema de archivos virtual de Linux. 72
El objeto superbloque. 75
El objeto nodo-i. 76
El objeto entradaD. 76
El objeto archivo. 76
El sistema de archivos de Windows. 77
Características de NTFS. 77

Referencias y bibliografía de la sección. 78

Definición de archivo y sus atributos.

Concepto de archivo.

Las computadoras pueden almacenar información en varios soportes de almacenamiento. Para que el sistema informático sea cómodo de utilizar, el sistema operativo proporciona una vista lógica uniforme para el almacenamiento de la información. El sistema operativo realiza una abstracción de las propiedades físicas de los dispositivos de almacenamiento, con el fin de definir una unidad lógica de almacenamiento, el ***archivo***.

Los archivos son mapeados por el sistema operativo sobre dispositivos físicos. Estos dispositivos de almacenamiento son, usualmente, no volátiles; es decir, que los contenidos persisten, aunque se produzcan fallos de alimentación o reinicios del Sistema Operativo.

Un archivo es una colección de información relacionada, con un nombre, que se graba en ***almacenamiento secundario***. Desde la perspectiva del usuario, un archivo es la unidad más pequeña de almacenamiento secundario lógico; en otras palabras, no pueden escribirse datos en el almacenamiento secundario a menos que estos se encuentren dentro un archivo.

En general, un archivo es una secuencia de bits, bytes, líneas o registros cuyo significado está definido por el creador y el usuario del archivo. Por tanto, el concepto de archivo es extremadamente general. La información contenida en un archivo es definida por su creador.

En un archivo puede almacenarse muchos tipos distintos de información: programas, datos numéricos, texto, imágenes gráficas, grabaciones sonoras, etc. Un archivo tiene una determinada estructura definida que dependerá de su tipo. Un archivo de texto es una secuencia de caracteres organizada en líneas (y posiblemente en páginas). Un archivo fuente es una secuencia de subrutinas y funciones. Un archivo ejecutable es una serie de secciones de código que el cargador puede cargar en memoria y ejecutar.

En la mayoría de las aplicaciones, el archivo es un elemento central. Es muy común que la entrada a una aplicación se realice mediante un archivo. También es habitual que las aplicaciones guarden la salida en un archivo para un almacenamiento a largo plazo o para su acceso posterior por parte del usuario u otros programas.

Los archivos tienen vida fuera de cualquier aplicación individual que los utilice como entrada y/o salida. Los usuarios desean poder acceder a los archivos, guardarlos y mantener la integridad de sus contenidos.

Como mínimo, un sistema de gestión de archivos necesita servicios especiales del sistema operativo; como máximo, un sistema de gestión de archivos completo se considera parte del sistema operativo.

Nomenclatura de archivos.

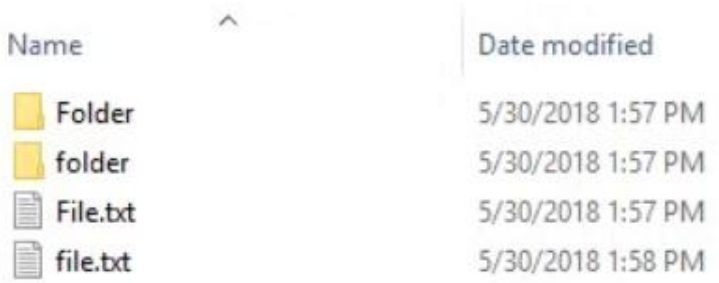
Los archivos son un mecanismo de ***abstracción***. Proporcionan una manera de almacenar información en la unidad de almacenamiento y leerla después. Esto se debe hacer de tal forma que se aísle al usuario de los detalles acerca de cómo y dónde se almacena la información y cómo funcionan las unidades de almacenamiento en realidad, ya sean discos, SSD, etc.

Probablemente, la característica más importante de cualquier mecanismo de abstracción sea la manera en que los objetos administrados son denominados. Cuando un proceso crea un archivo le proporciona un ***nombre***. Cuando el proceso termina, el archivo continúa existiendo y puede ser utilizado por otros procesos mediante su nombre.

Las reglas exactas para denominar archivos varían un poco de un sistema a otro, pero todos los sistemas operativos actuales permiten cadenas letras como nombres de archivos legales.

Por ende, *andrea*, *bruce* y *cathy* son posibles nombres de archivos. Con frecuencia también se permiten dígitos y caracteres especiales, por lo que nombres como *2*, *¡urgente!* y *Fig.2-14* son a menudo válidos también. Muchos sistemas de archivos admiten nombres de hasta 255 caracteres.

Algunos sistemas de archivos diferencian las letras mayúsculas de las minúsculas, mientras que otros no. UNIX cae en la primera categoría; MS-DOS en la segunda. Así, un sistema UNIX puede tener los siguientes nombres como tres archivos distintos: *maria*, *Maria* y *MARIA*. En MS-DOS, todos estos nombres se refieren al mismo archivo.



Windows 95 y Windows 98 utilizaban el sistema de archivos de MS-DOS conocido como ***FAT-16*** y por ende heredaban muchas de sus propiedades, como la forma en que se construyen sus nombres.

Windows 98 introdujo algunas extensiones a FAT-16, lo cual condujo a ***FAT-32***, pero estos dos sistemas son bastante similares. Además, Windows NT, Windows 2000, Windows XP y Windows Vista admitían ambos sistemas de archivos FAT, que en realidad ya son obsoletos.

Los cuatro sistemas operativos basados en NT tienen un sistema de archivos nativo (***NTFS***) con diferentes propiedades (como los nombres de archivos en ***Unicode***[1]).

[1] Unicode es un estándar de codificación de caracteres que define un número único para cada carácter, lo que permite la representación y el intercambio de texto en cualquier idioma y sistema de escritura. Es una especie de diccionario universal de caracteres, que incluye desde letras y números hasta símbolos.

Muchos sistemas operativos aceptan nombres de archivos en dos partes, separadas por un punto, como en *prog.c*. La parte que va después del punto se conoce como la ***extensión del archivo*** y por lo general indica algo acerca de su naturaleza.

En UNIX el tamaño de la extensión (si la hay) es a elección del usuario y un archivo puede incluso tener dos o más extensiones, como en *paginainicio.html.zip*, donde *.html* indica una página Web en HTML y *.zip* indica que el archivo se ha comprimido mediante el programa ***zip***.

Algunas de las extensiones de archivos más comunes y sus significados se muestran en la siguiente figura:

Extensión	Significado
archivo.bak	Archivo de respaldo
archivo.c	Programa fuente en C
archivo.gif	Imagen en Formato de Intercambio de Gráficos de CompuServe
archivo.hlp	Archivo de ayuda
archivo.html	Documento en el Lenguaje de Marcación de Hipertexto de World Wide Web
archivo.jpg	Imagen fija codificada con el estándar JPEG
archivo.mp3	Música codificada en formato de audio MPEG capa 3
archivo.mpg	Película codificada con el estándar MPEG
archivo.o	Archivo objeto (producido por el compilador, no se ha enlazado todavía)
archivo.pdf	Archivo en Formato de Documento Portable
archivo.ps	Archivo de PostScript
archivo.tex	Entrada para el programa formateador TEX
archivo.txt	Archivo de texto general
archivo.zip	Archivo comprimido

Ilustración 1 - Algunas extensiones de archivos comunes.

En algunos sistemas (como UNIX) las extensiones de archivo son sólo convenciones y no son impuestas por los sistemas operativos. Un archivo llamado *archivo.txt* podría ser algún tipo de archivo de texto, pero ese nombre es más un recordatorio para el propietario que un medio para transportar información a la computadora. Por otro lado, un compilador de C podría insistir que los archivos que va a compilar terminen con *.c* y podría rehusarse a compilarlos si no tienen esa terminación.

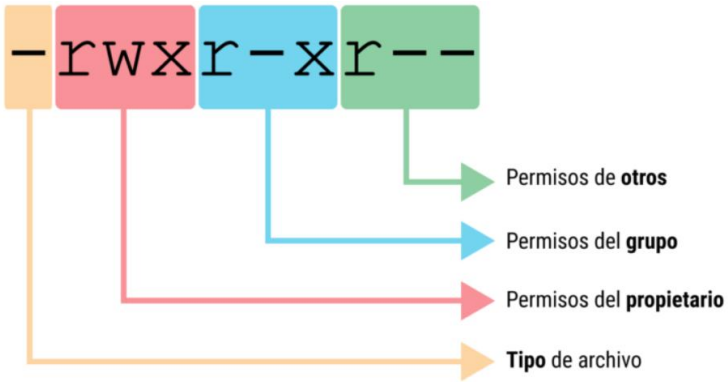
Las convenciones como ésta son especialmente útiles cuando el mismo programa puede manejar diferentes tipos de archivos. Por ejemplo, el compilador C puede recibir una lista de varios archivos para compilarlos y enlazarlos, algunos de ellos archivos de C y otros archivos de lenguaje ensamblador. Entonces, la extensión se vuelve esencial para que el compilador sepa cuáles son archivos de C, cuáles son archivos de lenguaje ensamblador y cuáles son archivos de otro tipo.

Caso contrario, Windows está consciente de las extensiones y les asigna significado. Los usuarios (o procesos) pueden registrar extensiones con el sistema operativo y especificar para cada una cuál programa “posee” esa extensión. Cuando un usuario hace doble clic sobre un nombre de archivo, el programa asignado a su extensión de archivo se inicia con el archivo como parámetro. Por ejemplo, al hacer doble clic en *archivo.doc* se inicia *Microsoft Word* con *archivo.doc* como el archivo inicial a editar.

Atributos de archivo.

Los atributos de un archivo varían de un sistema operativo a otro, pero típicamente son los siguientes:

- **Nombre**. El nombre de archivo simbólico es la única información que se mantiene en un formato legible por parte de las personas.
- **Identificador**. Esta etiqueta, que usualmente es un número, identifica el archivo dentro del sistema de archivos; se trata de la versión no legible por las personas del nombre del archivo.
- **Tipo.** Esta información es necesaria para los sistemas que soporten diferentes tipos de archivos.
- **Ubicación.** Esta información es un puntero a un dispositivo y a la ubicación del archivo dentro de dicho dispositivo.
- **Tamaño**. Este atributo expresa el tamaño actual del archivo (en bytes, palabras o bloques) y, posiblemente, el tamaño máximo permitido.
- **Protección**. Información de control de acceso que determina quién puede leer el archivo, escribir en el archivo, ejecutarlo, etc.
- **Fecha, hora e identificación del usuario.** Esta información puede mantenerse para los sucesos de creación, de última modificación y de último uso del archivo. Estos datos pueden resultar útiles para propósitos de protección, seguridad y monitorización del uso del archivo.



Más atributos de archivos.

A los atributos de archivo también se los conoce como ***metadatos***. La lista de atributos varía considerablemente de un sistema a otro. La tabla siguiente muestra algunas de las posibilidades, pero existen otras. Ningún sistema existente tiene todos, pero cada uno de ellos está presente en algún sistema.

Atributo	Significado
Protección	Quién puede acceso al archivo y en qué forma
Contraseña	Contraseña necesaria para acceder al archivo
Creador	ID de la persona que creó el archivo
Propietario	El propietario actual
Bandera de sólo lectura	0 para lectura/escritura; 1 para sólo lectura
Bandera oculto	0 para normal; 1 para que no aparezca en los listados
Bandera del sistema	0 para archivos normales; 1 para archivo del sistema
Bandera de archivo	0 si ha sido respaldado; 1 si necesita respaldarse
Bandera ASCII/binario	0 para archivo ASCII; 1 para archivo binario
Bandera de acceso aleatorio	0 para sólo acceso secuencial; 1 para acceso aleatorio
Bandera temporal	0 para normal; 1 para eliminar archivo al salir del proceso
Banderas de bloqueo	0 para desbloqueado; distinto de cero para bloqueado
Longitud de registro	Número de bytes en un registro
Posición de la llave	Desplazamiento de la llave dentro de cada registro
Longitud de la llave	Número de bytes en el campo llave
Hora de creación	Fecha y hora en que se creó el archivo
Hora del último acceso	Fecha y hora en que se accedió al archivo por última vez
Hora de la última modificación	Fecha y hora en que se modificó por última vez el archivo
Tamaño actual	Número de bytes en el archivo
Tamaño máximo	Número de bytes hasta donde puede crecer el archivo

Ilustración 2 - Algunos posibles atributos de archivos.

Los primeros cuatro atributos se relacionan con la protección del archivo e indican quién puede acceder a él y quién no.

En algunos sistemas, el usuario debe presentar una contraseña para acceder a un archivo, en cuyo caso la contraseña debe ser uno de los atributos.

Las banderas son bits o campos cortos que controlan o habilitan cierta propiedad específica. Por ejemplo, los archivos con la *bandera oculto* en 1 no aparecen en los listados de todos los archivos.

La *bandera de archivo* es un bit que lleva el registro de si el archivo se ha respaldado recientemente.

Operaciones con los archivos.

Un archivo es un tipo abstracto de datos. Para definir un archivo apropiadamente, debemos considerar las operaciones que pueden realizarse con los archivos. El sistema operativo puede proporcionar llamadas al sistema para ***crear***, ***escribir***, ***leer***, ***reposicionar***, ***borrar*** y ***truncar*** archivos.

Creación de un archivo. Para crear un archivo hace falta ejecutar dos pasos. En primer lugar, es necesario encontrar espacio para el archivo dentro del sistema de archivos. En segundo lugar, es necesario incluir en el directorio una entrada para el nuevo archivo.

Capa 1	**Función de Alto Nivel** (`fopen`)	**El Asistente:** Proporciona una interfaz simplificada para el programador y automatiza tareas de organización de datos.
Capa 2	**Wrapper Técnico** (`open` / `creat`)	**El Mensajero:** Actúa como interfaz intermedia para traducir los parámetros del lenguaje al formato que requiere el procesador.
Capa 3	**Llamada al Sistema** (`SYS_CREAT`)	**La Orden Real:** Punto de ejecución donde se solicita al núcleo del sistema operativo realizar una operación privilegiada.

El siguiente código muestra el uso básico del wrapper ***open()*** en C.

```c
#include <fcntl.h>
#include <unistd.h>

int fd = open("nuevo.txt", O_CREAT | O_WRONLY, 0644);
```

Ilustración 3 – Ejemplo de la creación de un archivo en C estilo POSIX.

Escritura en un archivo. Para escribir en un archivo, debemos realizar una llamada al sistema que especifique tanto el nombre del archivo como la información que hay que escribir en el archivo. Dado el nombre del archivo, el sistema explora el directorio para hallar la ubicación del archivo. El sistema debe mantener un puntero de escritura que haga referencia a la ubicación dentro del archivo en la que debe tener lugar la siguiente escritura. El puntero de escritura debe actualizarse cada vez que se escriba en el archivo nueva información.

```c
#include <fcntl.h>
#include <unistd.h>

int fd = open("datos.txt", O_WRONLY | O_CREAT, 0644);
write(fd, "Hola mundo\n", 11);
close(fd);
```

Ilustración 4 - Ejemplo de la creación y escritura en un archivo en C estilo POSIX.

1. `open()` → abre o crea el archivo, devolviendo un descriptor (`fd`).
2. `write()` → **escribe** datos desde el programa al archivo.
3. `close()` → cierra el archivo y asegura que se guarde todo.

Lectura de un archivo. Para leer desde un archivo, utilizamos una llamada al sistema que especifica el nombre del archivo y dónde debe colocarse (dentro de la memoria) el siguiente bloque del archivo. De nuevo, exploramos el directorio para hallar la entrada asociada y el sistema necesitará mantener un puntero de lectura que haga referencia a la ubicación dentro del archivo en la que tiene que tener lugar la siguiente lectura. Una vez que la lectura se ha completado, se actualiza el puntero de lectura. Puesto que los procesos, usualmente, están o bien leyendo del archivo o bien escribiendo en él, se puede almacenar la ubicación correspondiente a la operación actual en un puntero de posición actual del archivo que será diferente para cada proceso.

Las operaciones de lectura y escritura utilizan el mismo puntero, ahorrando así espacio y reduciendo la complejidad del sistema.

```c
#include <stdio.h>

int main() {
    FILE *archivo;
    char buffer[100];

    // Interfaz de librería estándar (C Standard Library)
    archivo = fopen("datos.txt", "r");

    if (archivo != NULL) {
        if (fgets(buffer, sizeof(buffer), archivo)) {
            printf("Contenido: %s", buffer);
        }
        fclose(archivo);
    }
    return 0;
}
```

Ilustración 5 - Lectura de un archivo con fopen() en C.

```c
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char buffer[100];

    // Wrapper técnico: Interfaz de enlace con el Sistema Operativo
    fd = open("datos.txt", O_RDONLY);

    if (fd != -1) {
        // Ejecución de lectura a través del descriptor de archivo
        int bytes = read(fd, buffer, sizeof(buffer) - 1);
        if (bytes > 0) {
            buffer[bytes] = '\0';
            write(1, buffer, bytes);
        }
        close(fd);
    }
    return 0;
}
```

Ilustración 6 - Lectura de un archivo con el wrapper open() en C.

Reposicionamiento dentro de un archivo. Se explora el directorio para hallar la correspondiente entrada y se reposiciona el puntero de posición actual dentro de un archivo, asignándole un nuevo valor. El reposicionamiento dentro del archivo no tiene por qué implicar que se realice ninguna operación de E/S. Esta operación de archivo se conoce también con el nombre de búsqueda en el archivo.

Borrado de un archivo. Para borrar un archivo, exploramos el directorio en busca del archivo indicado. Habiendo hallado la entrada de directorio asociada, liberamos todo el espacio del archivo, de modo que pueda ser reutilizado por otros archivos, y borramos también la propia entrada del directorio.

```c
#include <stdio.h>

int main() {
    if (remove("archivo.txt") == 0) { // Llamada a procedimiento
        printf("Archivo borrado correctamente.\n");
    } else {
        printf("Error al borrar archivo.\n");
    }
    return 0;
}
```

Ilustración 7 - Llamada a librería (remove) para eliminar un archivo.

Truncado de un archivo. El usuario puede querer borrar el contenido de un archivo, pero manteniendo sus atributos. En lugar de forzar al usuario a borrar el archivo y volverlo a crear, esta función permite que los atributos no se vean modificados (excepto la longitud del archivo), mientras que el archivo se reinicializa asignándole una longitud igual a cero y liberando el espacio que tuviera asignado.

Estas seis operaciones básicas forman el conjunto mínimo de operaciones de archivo requeridas.

Otras operaciones comunes son la adición de nueva información al final de un archivo existente y el renombrado de un archivo existente. Estas operaciones primitivas pueden a su vez combinarse para realizar otras operaciones de archivo. Por ejemplo, podemos crear una copia de un archivo, o copiar el archivo en otro dispositivo de E/S.

Referencias y bibliografía de la sección.

- Tanenbaum 4-1-1 / 4 -1-5.
- Silberschatz 10-1.
- Stallings pág. 548.

Tipos de archivos.

Muchos sistemas operativos soportan varios tipos de archivos. Por ejemplo, UNIX y Windows ***tienen archivos*** y ***directorios regulares***. UNIX también tiene ***archivos especiales de caracteres*** y ***de bloques***.

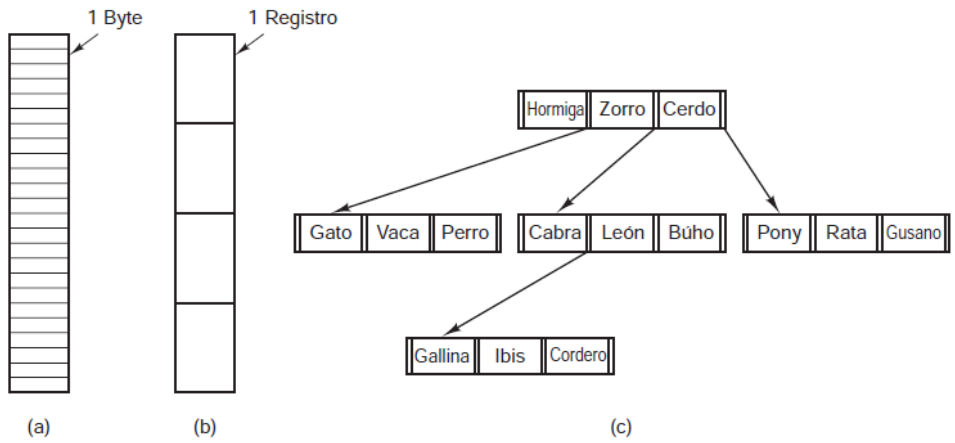


Ilustración 8 - Tres tipos de archivos. (a) Secuencia de bytes. (b) Secuencia de registros. (c) Árbol.

Los ***archivos regulares*** son los que contienen información del usuario. Todos los archivos de la figura anterior son archivos regulares.

Los ***archivos especiales de caracteres*** se relacionan con la entrada/salida y se utilizan para modelar dispositivos de E/S en serie, tales como terminales, impresoras y redes.

Característica	Detalle
Acceso	Secuencial
Operaciones típicas	`read()`, `write()`
Dispositivos comunes	Teclado, ratón, terminal
Ejemplo de uso	Leer caracteres de teclado y escribir en pantalla

Ilustración 9 - Características de los archivos de caracteres.

Los ***archivos especiales de bloques*** se utilizan para modelar discos.

Característica	Detalle
Acceso	Aleatorio y secuencial
Operaciones típicas	`read()`, `write()` con offsets
Dispositivos comunes	Disco duro, SSD
Ejemplo de uso	Leer un bloque de disco o escribir en un sector

Ilustración 10 - Características de los archivos de bloque.

Por lo general, los archivos regulares son archivos ***ASCII*** o ***binarios***. Los archivos ***ASCII*** consisten en líneas de texto. En algunos sistemas, cada línea se termina con un carácter de retorno de carro[2]. En otros se utiliza el carácter de avance de línea. Algunos sistemas (por ejemplo, MS-DOS) utilizan ambos. No todas las líneas necesitan ser de la misma longitud.

La gran ventaja de los archivos ASCII es que se pueden mostrar e imprimir como están, y se pueden editar con cualquier editor de texto. Además, si muchos programas utilizan archivos ASCII para entrada y salida, es fácil conectar la salida de un programa con la entrada de otro, como en las canalizaciones de shell. (La “plomería” entre procesos no es más fácil, pero la

[2] El "retorno de carro" (CR, de "carriage return") es un carácter de control que indica el final de una línea en un archivo de texto, o que mueve el cursor al principio de la línea actual. En términos sencillos, es como presionar la tecla "Enter" en un teclado para empezar una nueva línea.

interpretación de la información lo es si una convención estándar, tal como ASCII, se utiliza para expresarla).

Característica	Detalle
Formato	Texto plano
Contenido	Letras, números, símbolos y caracteres de control (salto de línea, tabulación, etc.)
Lectura/escritura	Secuencial, normalmente con funciones de texto (`fgets` , `fprintf` , etc.)
Compatibilidad	Muy alta, puede abrirse en cualquier sistema operativo
Extensiones típicas	`.txt` , `.csv` , `.log`

Ilustración 11 - Características de los archivos ASCII.

Otros archivos son ***binarios.*** Al imprimirlos en pantalla aparece un listado incomprensible de caracteres. Por lo general tienen cierta estructura interna conocida por los programas que los utilizan.

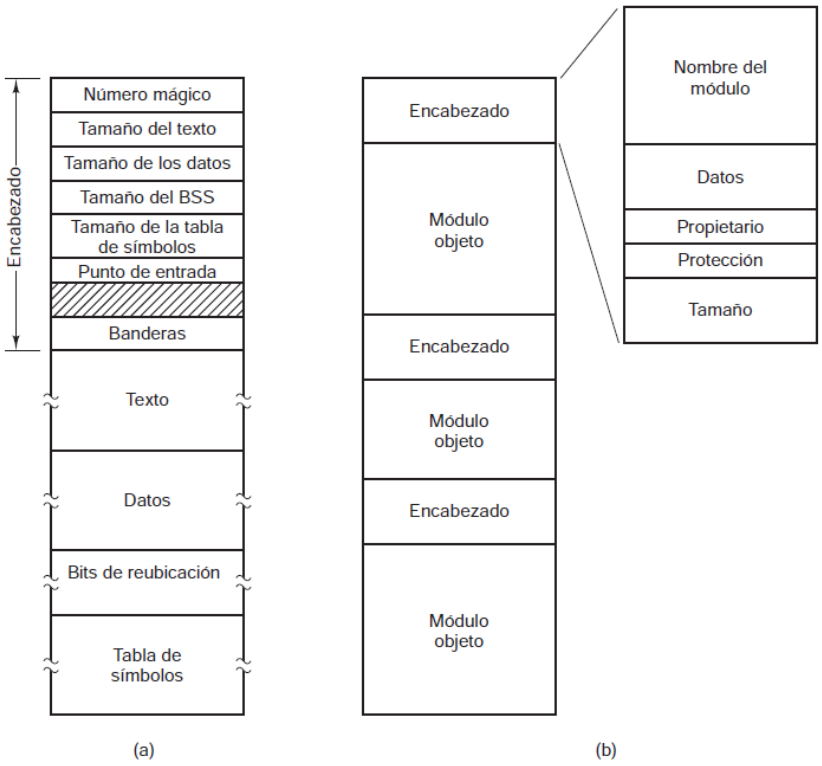


Ilustración 12 - (a) Un archivo ejecutable. (b) Un archivo.

Por ejemplo, en la figura anterior vemos un archivo binario ejecutable simple tomado de una de las primeras versiones de UNIX. Aunque técnicamente el archivo es sólo una secuencia de bytes, el sistema operativo sólo ejecutará un archivo si tiene el formato apropiado. Este archivo tiene cinco secciones: encabezado, texto, datos, bits de reubicación y tabla de símbolos.

El encabezado con un supuesto ***número mágico***[3], que identifica al archivo como un archivo ejecutable (para evitar la ejecución accidental de un archivo que no tenga este formato). Después vienen los tamaños de las diversas partes del archivo, la dirección en la que empieza la ejecución y algunos bits de bandera.

Luego del encabezado están el texto y los datos del programa en sí. Éstos se cargan en memoria y se reubican usando los bits de reubicación.

La tabla de símbolos se utiliza para depurar[4].

Nuestro segundo ejemplo (b), consiste en una colección de procedimientos (módulos) de biblioteca compilados, pero no enlazados. A cada uno se le antepone un encabezado que indica su nombre, fecha de creación, propietario, código de protección y tamaño.

Al igual que en el caso del archivo ejecutable, los encabezados de los módulos están llenos de números binarios. Al imprimirlos en pantalla se produciría basura como salida.

Cada sistema operativo debe reconocer por lo menos un tipo de archivo (su propio archivo ejecutable) y algunos reconocen más.

En términos de UNIX, el programa ***make*** había sido integrado al shell. Las extensiones de archivo eran obligatorias, por lo que el sistema operativo podía saber cuál programa binario se derivaba de cuál fuente.

Tener archivos fuertemente tipificados como éstos ocasiona problemas cada vez que el usuario hace algo que los diseñadores del sistema no esperaban. Por ejemplo, consideremos un sistema en el que los archivos de salida de un programa tienen la extensión *.dat* (archivos de datos). Si un usuario escribe un programa formateador que lea un archivo *.c* (programa de C), lo transforma (por ejemplo, convirtiéndolo a un esquema de sangría estándar) y después escribe el archivo transformado como salida, el archivo de salida será de tipo *.dat*. Si el usuario trata de ofrecer esto al compilador de C para que lo compile, el sistema se rehusará debido a que tienen la extensión incorrecta. Los intentos de copiar *archivo.dat* a *archivo.c* serán rechazados por el sistema como inválidos (para proteger al usuario contra los errores).

[3] Un "número mágico" (magic number) es una secuencia de bytes, generalmente al principio de un archivo, que identifica el tipo o formato del archivo de manera única. Estos números actúan como una firma que ayuda a las aplicaciones y al sistema operativo a reconocer y procesar correctamente el archivo.

[4] La información de la tabla de símbolos se puede utilizar para depurar el código, permitiendo identificar los nombres y ubicaciones de las variables. Además, durante la compilación, la tabla de símbolos ayuda a verificar que los símbolos se utilicen de manera correcta (por ejemplo, que una variable se use con el tipo de datos adecuado).

Reconocimiento del tipo de archivo.

Cuando diseñamos un sistema de archivos (de hecho, cuando diseñamos todo un sistema operativo) siempre debemos considerar si el sistema operativo debe reconocer y soportar el concepto de tipo de archivo. Si un sistema operativo reconoce el tipo de un archivo, podrá operar con ese archivo de formas razonables.

Una técnica común para implementar los tipos de archivo consiste en incluir el tipo como parte del nombre de archivo. El nombre se divide en dos partes: un nombre y una extensión, usualmente separadas por un carácter de punto. De esta forma, el usuario y el sistema operativo pueden determinar, con sólo analizar el nombre, cuál es el tipo de cada archivo.

El sistema utiliza la extensión para indicar el tipo del archivo y el tipo de operaciones que pueden realizarse con dicho archivo. Por ejemplo, sólo los archivos con extensión .com, .exe o .bat pueden ejecutarse.

Los archivos .com y .exe son dos tipos de archivos ejecutables binarios, mientras que un archivo .bat es un archivo de procesamiento por lotes que contiene, en formato ASCII, una serie de comandos dirigidos al sistema operativo.

Los programas de aplicación también utilizan las extensiones para indicar los tipos de archivo en los que están interesadas. Por ejemplo, los ensambladores esperan que los archivos fuente tengan una extensión .asm, y el procesador de textos Microsoft Word espera que sus archivos terminen con una extensión .doc.

Estas extensiones no siempre son obligatorias, por lo que un usuario podría especificar un archivo sin la extensión.

Tipo de archivo	Windows	Linux
Archivo de texto	documento.txt → abre con Notepad o editor asociado	documento.txt → abre con cualquier editor de texto
Ejecutable	programa.exe → ejecuta directamente; debe ser .exe	programa → ejecuta si tiene permiso +x; extensión no necesaria
Script	script.bat → ejecutable (archivo por lotes)	script.sh → ejecutable si tiene +x y shebang

El sistema UNIX utiliza un simple ***número mágico*** almacenado al principio de algunos archivos para indicar, aproximadamente, el tipo del archivo: programa ejecutable, archivo de procesamiento por lotes (o script de la shell), etc.

El siguiente código es un ejemplo práctico en Linux (C) que muestra como detectar si un archivo es de tipo ZIP por medio del número mágico.

```c
#include <stdio.h>   // Librería estándar: fopen, fread, fclose
#include <unistd.h>  // Para llamadas al sistema si se usa read/open (opcional)

int main() {

    FILE *f = fopen("archivo.zip", "rb"); // fopen: procedimiento de librería que
internamente hace open (llamada al sistema)

    unsigned char buffer[4];

    fread(buffer, 1, 4, f);  // fread: procedimiento que internamente hace read (llamada
al sistema)

    fclose(f);               // fclose: procedimiento que internamente hace close
(llamada al sistema)

    // Verificación del número mágico (todo es código de usuario)
    if (buffer[0] == 0x50 && buffer[1] == 0x4B &&
        buffer[2] == 0x03 && buffer[3] == 0x04) {
        printf("Este es un archivo ZIP\n");   // Llamada a procedimiento printf
    } else {
        printf("No es un archivo ZIP\n");    // Llamada a procedimiento printf
    }

    return 0;
}
```

No todos los archivos tienen números mágicos, por lo que la funcionalidad del sistema no puede basarse exclusivamente en esta información. UNIX tampoco almacena el nombre del programa que creó el archivo.

En UNIX sí que se permiten las sugerencias en forma de extensión del nombre del archivo, pero estas extensiones ni son obligatorias ni el sistema operativo depende de ellas; principalmente, su objetivo es ayudar a los usuarios a determinar el tipo de contenido del archivo. Esas extensiones pueden ser utilizadas o ignoradas por cada aplicación concreta, correspondiendo dicha decisión al programador de la aplicación.

Tipos de archivos en UNIX.

Si hablamos específicamente del sistema de archivos de UNIX, podemos distinguir seis tipos de archivos:

• **Regulares u ordinarios.** Contiene datos arbitrarios en cero o más bloques de datos. Los archivos regulares contienen información introducida por un usuario, una aplicación o una utilidad del sistema. El sistema de archivos no impone ninguna estructura interna a un archivo regular, sino que lo trata como una ristra de bytes.

• **Directorios.** Contiene una lista de nombres de archivos más punteros a nodos-i asociados (nodos índices), descritos posteriormente. Los directorios se organizan jerárquicamente. Los directorios son realmente archivos ordinarios con privilegios de protección de escritura especiales de tal forma que sólo el sistema de archivos puede escribirlos, mientras que los programas de usuario tienen acceso de lectura.

• **Especiales.** No contienen datos, sino que proporcionan un mecanismo para asociar dispositivos físicos a nombres de archivos. Se utilizan nombres de archivos para acceder a los dispositivos periféricos, tales como terminales e impresoras. Cada dispositivo de E/S se asocia con un archivo especial.

• **Tuberías con nombre.** Una tubería es una utilidad de comunicación entre procesos. Una tubería guarda en un buffer los datos de su entrada de forma que un proceso que lea de la salida de la tubería reciba los datos del mismo modo que si leyera de una cola FIFO.

• **Enlaces**. En esencia, un enlace es un nombre alternativo de archivo para un archivo existente.

• **Enlaces simbólicos.** Se trata de un archivo de datos que contiene el nombre del archivo al que enlaza.

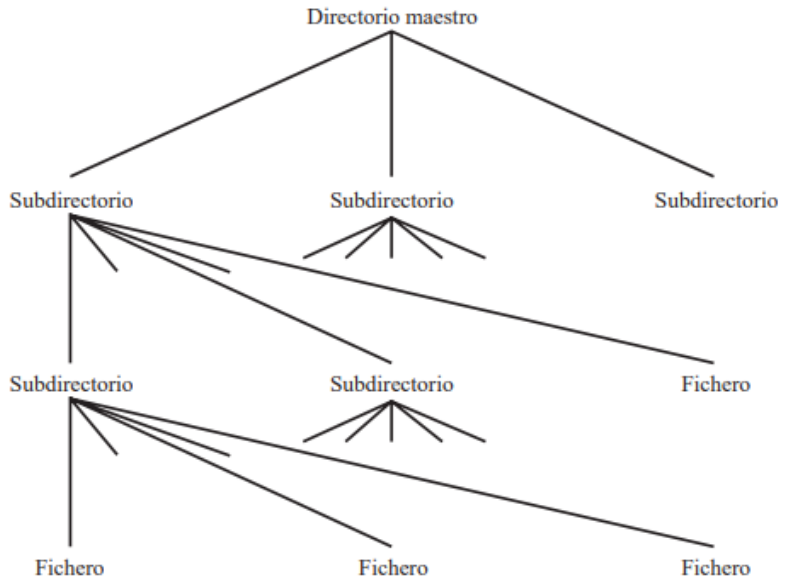


Ilustración 13 - Directorio estructurado en forma de árbol (estructura jerárquica).

Referencias y bibliografía de la sección.

- Tanenbaum 4-1-3.
- Silberschatz 10-1-3.

- Stallings pág. 574.

Tipos de acceso.

Algunos sistemas sólo proporcionan un método de acceso para los archivos, mientras que otros sistemas, soportan muchos métodos de acceso y elegir el método adecuado para cada aplicación concreta constituye uno de los principales problemas de diseño.

Acceso secuencial.

Los primeros sistemas operativos proporcionaban sólo un tipo de acceso: ***acceso secuencial***. En estos sistemas, un proceso podía leer todos los bytes o registros en un archivo en orden, empezando desde el principio, pero no podía saltar algunos y leerlos fuera de orden. Sin embargo, los archivos secuenciales podían rebobinarse para poder leerlos todas las veces que fuera necesario.

Los archivos secuenciales eran convenientes cuando el medio de almacenamiento era cinta magnética en vez de disco.

El secuencial es el método de acceso más simple. La información del archivo se procesa por orden, un registro después de otro.

Las lecturas y escrituras constituyen el grueso de las operaciones realizadas con un archivo. Una operación de lectura, lee la siguiente porción del archivo e incrementa automáticamente un puntero de archivo, que controla la ubicación de E/S. De forma similar, la operación de escritura, añade información al final del archivo y hace que el puntero avance hasta el final de los datos recién escritos (el nuevo final del archivo). Dichos archivos podrán reinicializarse para situar el puntero al principio de los mismos.

Aspecto	Descripción
Orden de lectura/escritura	De principio a fin (uno tras otro).
Eficiencia	Muy eficiente si se procesan **todos los registros** en orden.
Búsqueda	Lenta si se necesita un registro intermedio o final.
Uso común	Procesos por lotes, reportes, respaldos, archivos de log.

Acceso aleatorio o directo.

Cuando se empezó a usar discos para almacenar archivos, se hizo posible leer los bytes o registros de un archivo fuera de orden, pudiendo acceder a los registros por llave en vez de posición.

Los archivos cuyos bytes o registros se pueden leer en cualquier orden se llaman ***archivos de acceso aleatorio.***

Un archivo está compuesto de registros lógicos de longitud fija que permiten a los programas leer y escribir registros rápidamente, sin ningún orden concreto. El método de acceso directo o aleatorio se basa en un modelo de archivos que se corresponde con los dispositivos de disco, ya que los discos permiten el acceso aleatorio a cualquier bloque de un archivo.

Para el acceso directo, el archivo se considera como una secuencia numerada de bloques o registros. Por tanto, podemos leer el bloque 14, luego leer el bloque 53 y luego escribir el bloque 7. No existe ninguna restricción en cuanto al orden de lectura o escritura en los archivos de acceso aleatorio.

Los archivos de acceso aleatorio son esenciales para muchas aplicaciones, como los sistemas de bases de datos. Cuando se recibe una consulta relativa a un tema concreto, se calcula qué bloque contiene la respuesta y luego se lee ese bloque directamente para proporcionar la información deseada.

Por ejemplo, si el cliente de una aerolínea llama y desea reservar un asiento en un vuelo específico, el programa de reservación debe poder tener acceso al registro para ese vuelo sin tener que leer primero los miles de registros de otros vuelos.

Es posible utilizar dos métodos para especificar dónde se debe empezar a leer. En el primero, cada operación ***read*** da la posición en el archivo en la que se va a empezar a leer. En el segundo se provee una operación especial (***seek***) para establecer la posición actual. Después de una operación seek, el archivo se puede leer de manera secuencial desde la posición actual. Este último método se utiliza en UNIX y Windows.

Aspecto	Descripción
Orden de acceso	Cualquier registro, en cualquier orden.
Eficiencia	Muy rápida para búsqueda o actualización puntual.

Referencias y bibliografía de la sección.

- Tanenbaum 4-1-4.
- Silberschatz 10-2.

Estructura de un archivo.

Los archivos se pueden estructurar en una de varias formas. Tres posibilidades comunes se describen en la figura siguiente.

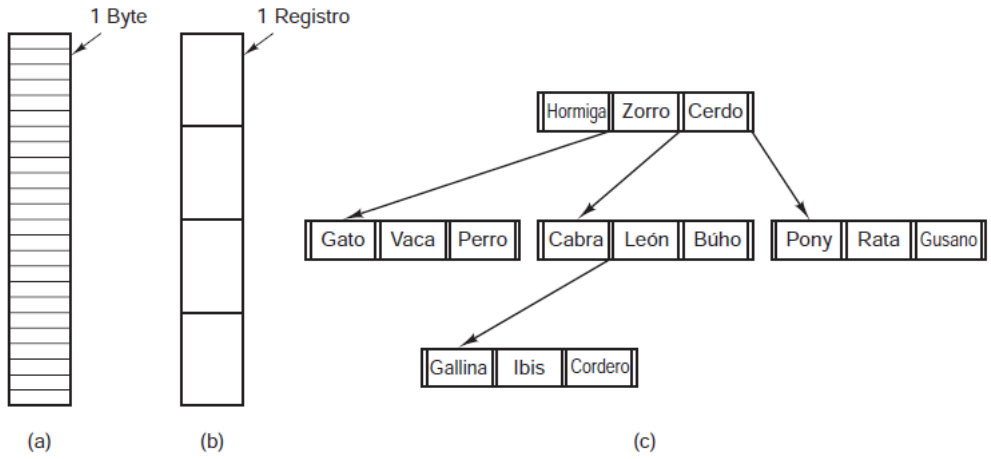


Ilustración 14 - Tres tipos de archivos. (a) Secuencia de bytes. (b) Secuencia de registros. (c) Árbol.

El archivo en *a* de la figura anterior es una secuencia de bytes sin estructura: el sistema operativo no sabe, ni le importa, qué hay en el archivo. Todo lo que ve son bytes. Cualquier significado debe ser impuesto por los programas a nivel usuario. Tanto UNIX como Windows utilizan esta metodología.

Hacer que el sistema operativo considere los archivos sólo como secuencias de bytes provee la máxima flexibilidad. Los programas de usuario pueden colocar cualquier cosa que quieran en sus archivos y denominarlos de cualquier manera conveniente. El sistema operativo no ayuda, pero tampoco estorba. Para los usuarios que desean realizar cosas inusuales, esto último puede ser muy importante. Todas las versiones de UNIX, MS-DOS y Windows utilizan este modelo de archivos.

Simplicidad del Kernel	El SO solo necesita implementar un puñado de **Syscalls** genéricas (`read`, `write`, `lseek`). No necesita código específico para cada tipo de archivo.
Evolución del Software	Se pueden crear nuevos formatos de archivos (como pasó con el MP3 o el MP4) sin necesidad de actualizar o parchear el Sistema Operativo.

El sistema operativo UNIX define todos los archivos como simples flujos de bytes. Cada byte es direccionable de manera individual, a partir de su desplazamiento con respecto al principio (o

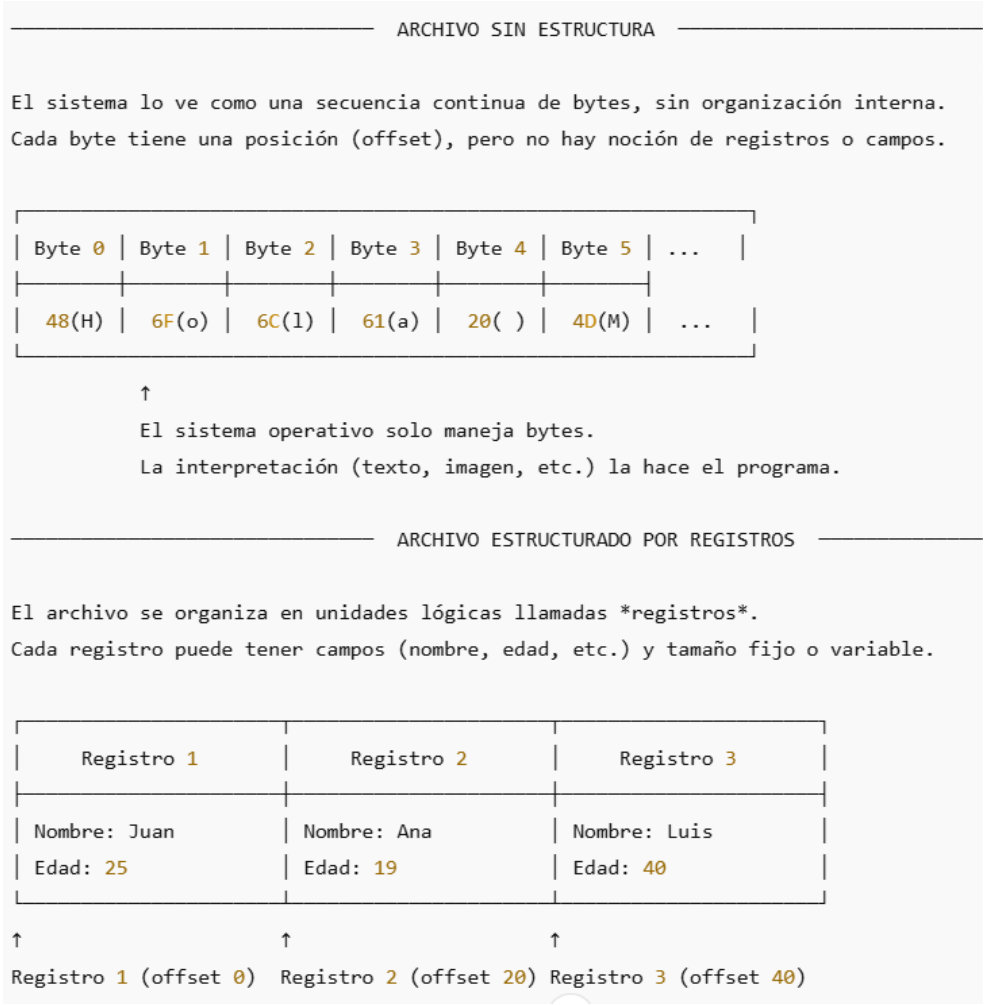
al final) del archivo. En este caso, el tamaño de registro lógico[5] es un byte. El sistema de archivos empaqueta y desempaqueta automáticamente los bytes en los bloques físicos del disco (por ejemplo, 512 bytes por bloque) según sea necesario.

El tamaño del registro lógico, el tamaño del bloque físico y la técnica de empaquetamiento determinan cuantos registros lógicos se almacenarán en cada bloque físico. El empaquetamiento puede ser realizado por el programa de aplicación del usuario o por el sistema operativo.

Característica	Descripción
Unidad básica:	Byte. El sistema ve el archivo como una secuencia de bytes numerados desde 0.
Sin estructura lógica interna:	No hay noción de "registro", "campo", "línea", etc.
Acceso:	Puede ser **secuencial** (leer desde el principio hasta el final) o **directo** (ir a un byte específico).
Gestión del SO:	El sistema operativo solo mantiene **metadatos** (nombre, tamaño, permisos, fechas, ubicación en disco), no estructura del contenido.
Interpretación:	Depende **exclusivamente del programa** que lo lea o escriba.

En el modelo de la parte *b*, un archivo es una secuencia de ***registros*** de longitud fija, cada uno con cierta estructura interna. El concepto central para la idea de que un archivo sea una secuencia de registros es la idea de que la operación de lectura devuelva un registro y la operación de escritura sobrescriba o agregue un registro.

[5] Un registro lógico en un archivo es una colección de campos relacionados que representan una unidad de información coherente. Estos campos pueden ser de diferentes tipos (números, caracteres, etc.) y su estructura define cómo se organiza la información dentro del registro.



El tercer tipo de estructura de archivo se muestra en la parte *c* de la figura anterior. En esta organización, un archivo consiste de un ***árbol de registros***, donde no todos son necesariamente de la misma longitud; cada uno de ellos contiene un campo ***llave*** en una posición fija dentro del registro. El árbol se ordena con base en el ***campo llave*** para permitir una búsqueda rápida por una llave específica.

La operación básica aquí no es obtener el "siguiente" registro, aunque eso también es posible, sino obtener el registro con una ***llave específica***.

En este caso, podríamos pedir al sistema que, por ejemplo, obtenga el registro cuya llave sea *pony*, sin preocuparnos acerca de su posición exacta en el archivo. Además, se pueden agregar nuevos registros al archivo, con el sistema operativo, y no el usuario, decidiendo dónde colocarlos.

Evidentemente, este tipo de archivos es bastante distinto de los flujos de bytes sin estructura que se usan en UNIX y Windows, pero se utiliza de manera amplia en las grandes computadoras mainframe que aún se emplean en algún procesamiento de datos comerciales.

Además, aunque sistemas operativos de propósito general (como Windows, Linux o macOS) no suelen usar ***árboles de registros*** para los archivos de los usuarios, sí que utilizan la filosofía de estos árboles de forma masiva "detrás de escena" para gestionar el disco.

NTFS (Windows) utiliza un ***árbol B+***[6] para organizar los directorios. Cuando busca un archivo por nombre, el Kernel navega por este árbol de registros.

Tipo	Unidad	Estruct.	Ejemplo	Ventaja	Desventaja
Sin estructura	Byte	Ninguna	.bin, .txt	Simple	Difícil leer
Por registros	Registro	Lineal	.dat, COBOL	Acceso directo	Poco flexible
En árbol	Nodo	Jerárq.	.xml, .json	Relac. complejas	Acceso lento

Referencias y bibliografía de la sección.

- Tanenbaum 4-1-2.
- Silberschatz 10-1-4 / 10-1-5.

Concepto de partición y su formato.

Estructuras de almacenamiento.

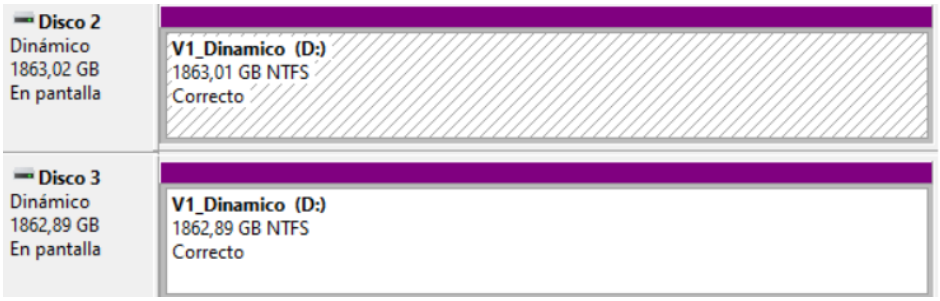
Un disco (o cualquier dispositivo de almacenamiento que sea lo suficientemente grande) puede utilizarse completamente para un sistema de archivos. Sin embargo, en ciertas ocasiones es deseable colocar múltiples sistemas de archivos en un mismo disco o utilizar partes de un disco para un sistema de archivos y otras partes para otras cosas, como por ejemplo para espacio de ***intercambio*** o como espacio de disco sin formato (raw).

Estas partes se conocen con diversos nombres, como ***particiones***, franjas o (en el mundo IBM) minidiscos.



[6] Un árbol B+ es una estructura de datos de árbol balanceada que permite búsquedas, inserciones y eliminaciones eficientes en grandes conjuntos de datos.

Podemos crear un sistema de archivos en cada una de estas partes del disco. Las partes también pueden combinarse para formar estructuras de mayor tamaño, conocidas con el nombre de ***volúmenes***, y también pueden crearse sistemas de archivos en dichos volúmenes.



Por el momento, utilizaremos el término volumen simplemente para referirnos a un espacio de almacenamiento que alberga un sistema de archivos. Cada volumen puede considerarse como si fuera un disco virtual. Los volúmenes pueden también almacenar múltiples sistemas operativos, permitiendo que un sistema se inicie en cualquiera de ellos y ejecute dicho sistema operativo.

Cada volumen que contenga un sistema de archivos debe también contener información acerca de los archivos almacenados en el sistema. Esta información se almacena como entrada en un ***directorio de dispositivo***, también llamada **tabla de contenidos del volumen** o simplemente ***directorio***.

Sistema de archivos	Estructura equivalente
FAT (DOS/Windows)	FAT Table + Root Directory
NTFS (Windows)	MFT (Master File Table)
ext4 (Linux)	Inodos + directorios
ISO9660 (CD/DVD)	Directory Records

Esta tabla almacena información de todos los archivos de dicho volumen, como por ejemplo el nombre, la ubicación, el tamaño y el tipo. La figura siguiente muestra una organización típica de un sistema de archivo.

Se utilizan varias ***estructuras de datos*** en disco y en memoria para implementar un sistema de archivos. Estas estructuras de datos varían dependiendo del sistema operativo y del sistema de archivos, aunque hay algunos principios básicos que son de aplicación general.

En el disco, el sistema de archivos puede contener información acerca de cómo iniciar un sistema operativo que esté almacenado allí, acerca del número total de bloques, del número y la ubicación de los bloques libres, de la estructura de directorios y de los archivos individuales.

Algunas posibles estructuras son:

- Un ***bloque de control de arranque*** o ***BCB*** (por cada volumen) puede contener la información que el sistema necesita para iniciar un sistema operativo a partir de dicho volumen. Si el disco no contiene un sistema operativo, este bloque puede estar vacío. Normalmente, es el primer bloque del volumen. En UFS[7], se denomina bloque de inicio. En NFTS, se denomina sector de arranque de la partición.
- Un ***bloque de control de volumen*** o ***VCB*** (por cada volumen) contiene detalles acerca del volumen (o partición), tales como el número de bloques que hay en la partición, el tamaño de los bloques, el número de bloques libres y los punteros de bloques libres, así como un contador de bloques de información FCB libres y punteros FCB. En UFS, esto se denomina ***superbloque***, en NFTS, esta información se almacena en la ***tabla maestra de archivos***.
- Se utiliza una estructura de directorios por cada sistema de archivos. En UFS, esto incluye los nombres de los archivos y los nombres de inodo asociados. En NFTS, esta información se almacena en la tabla maestra de archivos.
- Un bloque FTB por cada archivo contiene numerosos detalles acerca del archivo, incluyendo los permisos correspondientes, el propietario, el tamaño y la ubicación de los bloques de datos. En UFS, esta estructura se denomina ***inodo***. En NFTS, esta información se almacena dentro de la tabla maestra de archivos, que utiliza una estructura de base de datos relacional, con una fila por cada archivo.

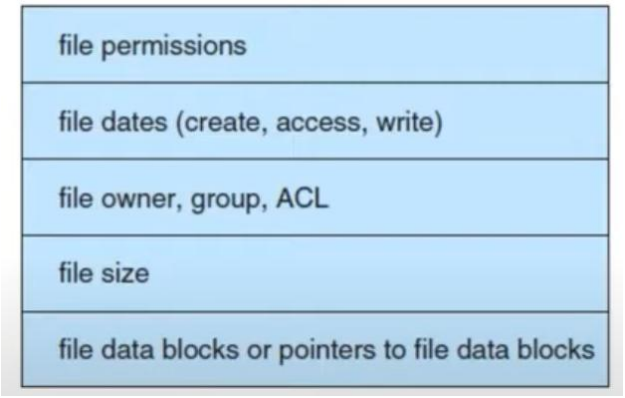
BCB	**Bloque de Inicio**	Contiene el código para la **carga y ejecución** del Sistema Operativo.
VCB	**Estado del Volumen**	Almacena los datos globales: capacidad total y registro de bloques libres.
Directorio	**Tabla de Nombres**	Vincula el nombre del archivo con su identificador interno.
FCB	**Metadatos del Archivo**	Contiene los detalles específicos: permisos, dueño y ubicación de los datos.

La información almacenada en memoria se utiliza tanto para la gestión de un sistema de archivos como para la mejora del rendimiento mediante mecanismos de caché. Los datos se cargan en el momento del montaje y se descartan cuando el dispositivo se desmonta. Las estructuras existentes pueden incluir las que a continuación se describen:

[7] Unix File System (UFS) es un sistema de archivos utilizado por varios sistemas operativos UNIX y POSIX.

- Una tabla de montaje en memoria contiene información acerca de cada volumen montado.
- Una caché de la estructura de directorios en memoria almacena la información relativa a los directorios a los que se ha accedido recientemente (para los directorios en los que hay que montar los volúmenes, puede contener un puntero a la tabla del volumen).
- La tabla global de archivos abiertos contiene una copia del FCB de cada archivo abierto, además de otras informaciones.
- La tabla de archivos abiertos de cada proceso contiene un puntero a la entrada apropiada de la entrada global de archivos abiertos, así como otras informaciones adicionales.

Para crear un nuevo archivo, un programa de aplicación llama al sistema lógico de archivos. El sistema lógico de archivos conoce el formato de las estructuras de directorio y, para crear un nuevo archivo, asigna un nuevo FCB (alternativamente, si la implementación del sistema de archivos crea todos los bloques FCB en el momento de creación del sistema de archivos, se asignará un FCB a partir del conjunto de bloques FCB libres). El sistema lee entonces el directorio apropiado en la memoria, lo actualiza con el nuevo nombre de archivo y el nuevo FCB y lo vuelve a escribir en el disco. En la siguiente figura se muestra un FCB típico.



Algunos sistemas operativos, incluyendo UNIX, tratan a los directorios exactamente igual que a los archivos, salvo porque se tratará de un archivo cuyo campo de tipo indicará que es un directorio.

Otros sistemas operativos, incluyendo Windows, poseen llamadas al sistema diferentes para los archivos y los directorios, y tratan a los directorios como entidades distintas de los archivos.

Independientemente de estas consideraciones estructurales de mayor nivel, el sistema lógico de archivos puede llamar al módulo de organización de archivos para mapear las operaciones de E/S de directorio sobre los números de bloque del disco, que se pasarán a su vez al sistema básico de archivos del sistema de control de E/S.



Para poder utilizar un archivo es necesario abrirlo. La llamada ***open ()*** pasa un nombre de archivo al sistema de archivos. La llamada al sistema open () primero busca en la tabla global de archivos abiertos para ver si el archivo está siendo ya utilizado por otro proceso. En caso afirmativo, se crea una entrada en la tabla de archivos abiertos del proceso que apunte a la tabla global de archivos abiertos existente. Este algoritmo puede ahorrarnos una gran cantidad de trabajo. Cuando un archivo está abierto, se busca en la estructura de directorios para encontrar el nombre de archivo indicado.

Usualmente, se almacenan parte de la estructura de directorio en una caché de memoria para acelerar las operaciones de directorio. Una vez encontrado el archivo, el FCB se copia en la tabla global de archivos abiertos existente en la memoria. Esta tabla no sólo almacena el FCB, sino que también controla el número de procesos que han abierto el archivo.

A continuación, se crea una entrada en la tabla de archivos abiertos del proceso, con un puntero a la entrada de la tabla global de archivos abiertos y algunos otros campos de información.

Estos otros campos de información pueden incluir un puntero a la ubicación actual dentro del archivo (para la siguiente operación ***read ()*** o ***write ()***) y el modo de acceso en que se ha abierto el archivo. La llamada ***open ()*** devuelve un puntero a la entrada apropiada de la tabla de archivos abiertos del proceso; todas las operaciones de archivo se realizan a partir de ahí mediante este puntero.

El nombre del archivo puede no formar parte de la tabla de archivos abiertos, ya que el sistema no lo utiliza para nada una vez que se ha localizado el FCB apropiado en el disco. Sin

embargo, se lo puede almacenar en caché, para ahorrar tiempo en las subsiguientes aperturas del mismo archivo. El nombre que se proporciona a esas entradas varía de unos sistemas a otros. En los sistemas UNIX se utiliza el término inglés ***file descriptor***, mientras que Windows prefiere el término file ***handle***; en castellano, utilizamos el término descriptor de archivo. En consecuencia, hasta que se cierre el archivo, todas las operaciones con el archivo se llevan a cabo mediante el descriptor de archivo contenido en la tabla de archivos abierto.

Cuando un proceso cierra el archivo, se elimina la entrada de la tabla del proceso y se decrementa el contador de aperturas existente en la tabla global. Cuando todos los usuarios que hayan abierto el archivo lo cierren, los metadatos actualizados se copian en la estructura de directorio residente en el disco y se elimina la entrada de la tabla global de archivos abiertos.

No deben infravalorarse los aspectos de almacenamiento en caché de las estructuras de los sistemas de archivos. La mayoría de los sistemas mantienen en la memoria toda la información acerca de un archivo abierto, salvo los propios bloques de datos. El sistema ***BSD UNIX*** resulta bastante típico en su uso de cachés para tratar de ahorrar operaciones de E/S de disco. Su tasa media de aciertos de caché, que es del 85 por ciento, muestra que merece la pena implementar estas técnicas.

La figura siguiente resume las estructuras de operaciones utilizadas en la implementación de un sistema de archivos.

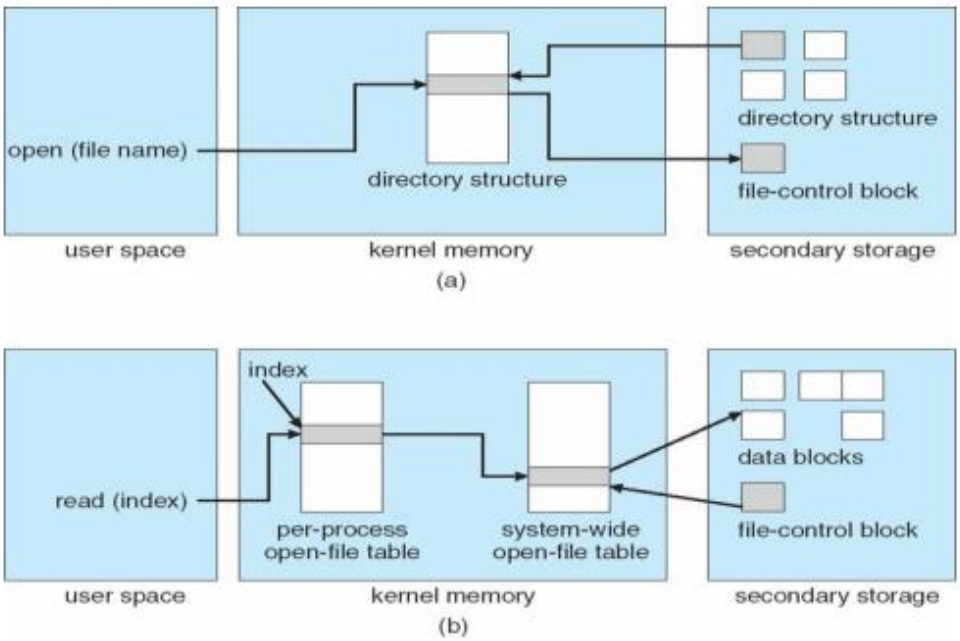


Ilustración 15 - Estructuras en memoria de un sistema de archivos. (a) Apertura de un archivo. (b) Lectura de un archivo.

Particiones y montaje.

La disposición de la información en un disco puede tener múltiples variantes, dependiendo del sistema operativo. Un disco puede dividirse en múltiples particiones, o bien un volumen puede abarcar múltiples particiones en múltiples discos.

Cada partición puede ser una partición “en bruto”, sin formato, que no contenga ningún sistema de archivos; o una partición “procesada”, que sí contenga un sistema de archivos.

Los discos sin formato se utilizan en aquellos casos en que no resulte apropiado emplear un sistema de archivos. Por ejemplo, el espacio de intercambio en UNIX puede utilizar una partición sin formato, ya que ese espacio de intercambio usa su propio formato en el disco y no emplea un sistema de archivos. De la misma forma, algunas bases de datos utilizan discos sin formato y formatean los datos de la forma que mejor se adapte a sus necesidades. Los discos sin formato también pueden almacenar la información que necesitan los sistemas de discos RAID[8], como los mapas de bits que indican qué bloques están ubicados en espejo y cuáles han sido modificados y necesitan ser duplicados.

De forma similar, los discos sin formato pueden contener una base de datos en miniatura que almacene la información de configuración RAID, como por ejemplo la información que indique qué discos son miembro de cada conjunto RAID.

La información de arranque puede almacenarse en una partición independiente. De nuevo, esa partición tiene su propio formato, porque en el momento del inicio el sistema no ha cargado aún los controladores de dispositivo para los sistemas de archivos y no puede, por tanto, interpretar el formato de ningún sistema de archivos. En lugar de ello, la información de arranque es, usualmente, una serie secuencial de bloques, que se cargan como una imagen binaria en la memoria. La ejecución de esa imagen binaria comienza en una ubicación predefinida, como por ejemplo el primer byte. Esta imagen de arranque puede contener más información, a parte de las instrucciones relativas a cómo iniciar un sistema operativo específico. Por ejemplo, en un PC y en otros sistemas, la máquina puede ser de arranque dual, pudiendo instalarse a múltiples sistemas operativos en dichos tipos de sistemas.

Podemos almacenar en el espacio de arranque un cargador de arranque que sea capaz de comprender múltiples sistemas de archivos y múltiples sistemas operativos; una vez cargado, ese cargador de arranque puede iniciar uno de los sistemas operativos disponibles en el disco.

El disco puede tener múltiples particiones, cada una de las cuales contendrá un tipo diferente de sistema de archivos y un sistema operativo distinto.

La partición raíz, que contiene el kernel del sistema operativo y, en ocasiones, otros archivos del sistema, se monta en el momento del arranque. Otros volúmenes pueden montarse automáticamente durante el arranque o pueden ser montados manualmente en algún momento posterior, dependiendo del sistema operativo.

Como parte de una operación de montaje, el sistema operativo verifica que el dispositivo contenga un sistema de archivos válido. Para realizar esa verificación, el sistema operativo pide al controlador del dispositivo que lea el directorio del dispositivo y verifique que este directorio tiene el formato esperado. Si el formato no es válido, será necesario comprobar y posiblemente corregir la coherencia de la partición, con o sin intervención del usuario.

Finalmente, el sistema operativo registrará en su tabla de montaje de la memoria que se ha montado un sistema de archivos, indicando el tipo de sistema de archivos de que se trata. Los detalles de esta función dependerán del sistema operativo. Los sistemas Microsoft Windows montan cada volumen en un espacio de nombres separado, que se designa mediante una letra

[8] RAID (Redundant Array of Independent Disks) se refiere a un sistema de almacenamiento que utiliza múltiples discos duros para mejorar la velocidad, redundancia y capacidad de almacenamiento.

y un carácter de dos puntos. Por ejemplo, para registrar que un sistema de archivos está montado en F:, el sistema operativo coloca un puntero al sistema de archivos dentro de un campo de la estructura de dispositivo correspondiente a F:. Cuando un proceso especifica dicha letra de unidad, el sistema operativo localiza el puntero apropiado al sistema de archivos y recorre las estructuras de directorio de dicho dispositivo hasta encontrar el archivo o directorio especificado. Las versiones más recientes de Windows pueden montar el sistema de archivos en cualquier punto dentro de la estructura de directorios existentes.

En UNIX, los sistemas de archivos pueden montarse en cualquier directorio. El montaje se implementa configurando un indicador dentro de la copia en memoria del inodo correspondiente a dicho directorio. Ese indicador atestigua que el directorio es un punto de montaje. Otro campo apunta a una entrada dentro de la tabla de montaje, que indica qué dispositivo está montado allí.

La entrada de la tabla de montaje contiene un puntero al superbloque del sistema de archivos existente en dicho dispositivo. Este esquema permite que el sistema operativo recorra su estructura de directorios, conmutando transparentemente entre sistemas de archivos de diferentes tipos.

Estructura del volumen en UNIX.

Un sistema de archivos UNIX reside en un único disco lógico o partición de disco y se compone de los siguientes elementos:

- **Bloque de arranque.** Contiene el código requerido para arrancar el sistema operativo.
- **Superbloque.** Contiene atributos e información sobre el sistema de archivos, tal como el tamaño de la partición y el tamaño de la tabla de nodos-i.
- **Tabla de nodos-i.** La colección de nodos-i para cada archivo.
- **Bloques de datos.** El espacio de almacenamiento disponible para los archivos de datos y subdirectorios.

Referencias y bibliografía de la sección.

- Silberschatz 10-3-1 / 11-2-1 / 11-2-2.
- Stallings pág. 574.

Esquemas de asignación de bloques.

Uno de los aspectos más críticos en la implementación del almacenamiento de archivos es el seguimiento y la asignación de los bloques de disco que pertenecen a cada archivo. Es decir, mantener un registro acerca de qué bloques de disco van con cuál archivo.

Para esto, los sistemas operativos utilizan diversos métodos de asignación.

Método	Disposición	Ventaja	Desventaja
Contigua	Consecutiva	Velocidad	Rigidez
Ligada	Encadenada	Eficiencia	Lentitud
FAT	Tabla central	Saltos	Fragilidad
Indexada	Bloque mapa	Flexibilidad	Tope de tamaño

Asignación contigua.

El esquema de asignación más simple es almacenar cada archivo como una serie contigua de bloques de disco. Así, en un disco con bloques de 1 KB, a un archivo de 50 KB se le asignarían 50 bloques consecutivos. Con bloques de 2 KB, se le asignarían 25 bloques consecutivos.

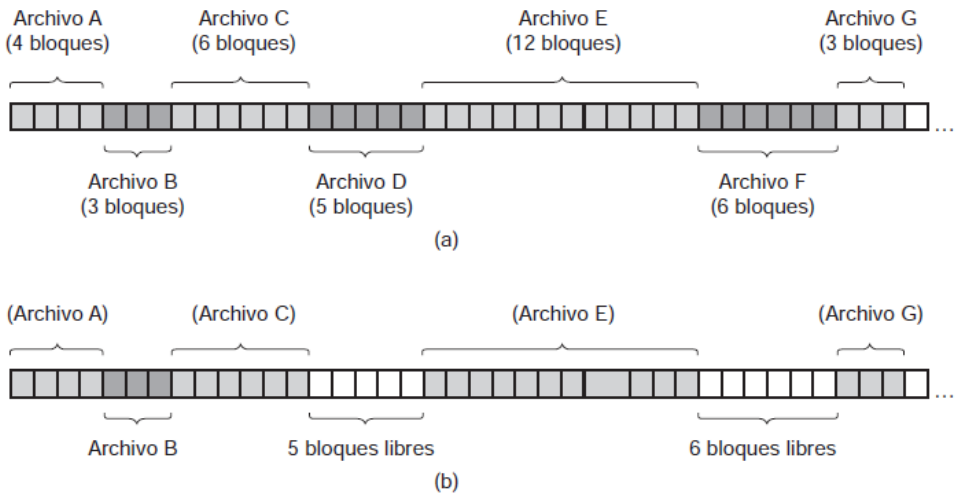


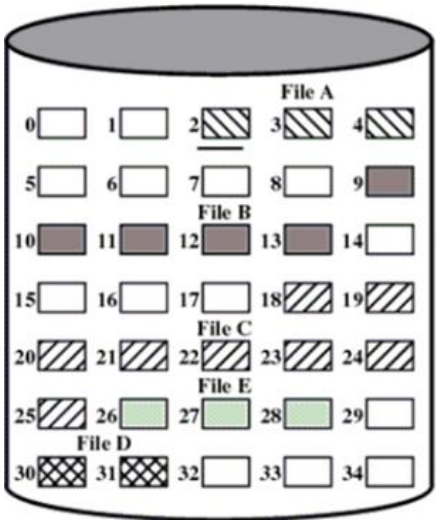
Ilustración 16 - (a) Asignación contigua de espacio de disco para siete archivos. (b) El estado del disco después de haber removido los archivos D y F.

En la parte *a* de la figura anterior podemos ver un ejemplo de asignación de almacenamiento contigua. Acá se muestran los primeros 40 bloques de disco, empezando con el bloque 0, a la izquierda. Al principio el disco estaba vacío, después se escribió un archivo *A* de cuatro bloques de longitud al disco, empezando desde el principio (bloque 0). Posteriormente se escribió un archivo de seis bloques llamado *B*, empezando justo después del archivo *A*.

Observemos que cada archivo empieza al inicio de un nuevo bloque, por lo que si el archivo *A* fuera realmente de 3 1/2 bloques, se desperdiciaría algo de espacio al final del último bloque.

En la figura se muestra un total de siete archivos, cada uno empezando en el bloque que va después del final del archivo anterior. Se utiliza sombreado sólo para facilitar la distinción de cada archivo. No tiene un significado real en términos de almacenamiento.

La asignación de espacio en disco contiguo tiene dos ventajas significativas. En primer lugar, es simple de implementar, ya que llevar un registro de la ubicación de los bloques de un archivo se reduce a recordar dos números: la dirección de disco del primer bloque y el número de bloques en el archivo. Dado el número del primer bloque, se puede encontrar el número de cualquier otro bloque con una simple suma.



Nombre de Archivo	Bloque inicial	Longitud
Archivo A	2	3
Archivo B	9	5
Archivo C	18	8
Archivo D	30	2
Archivo E	26	3

En segundo lugar, el rendimiento de lectura es excelente debido a que el archivo completo se puede leer del disco en una sola operación. Sólo se necesita una búsqueda (para el primer bloque).

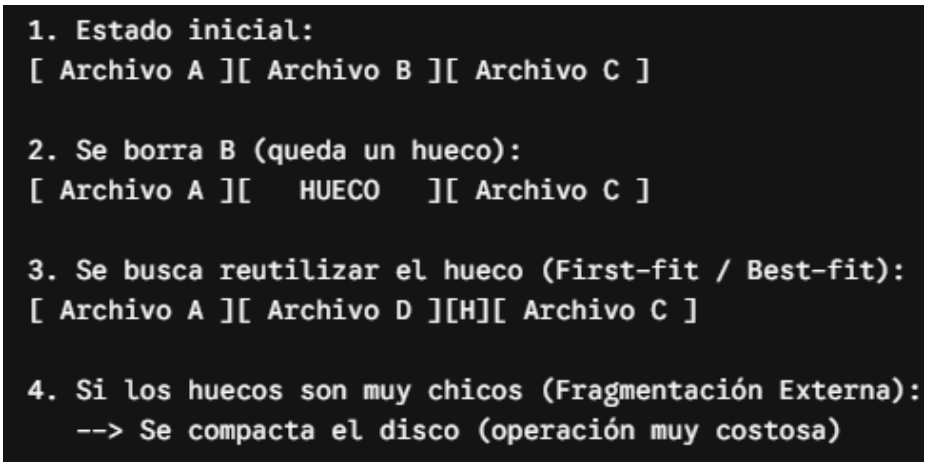
Después de eso, no son necesarias más búsquedas ni retrasos por rotación, por lo que los datos llegan con el ancho de banda completa del disco. Por ende, la asignación contigua es simple de implementar y tiene un alto rendimiento.

Por desgracia, la asignación contigua también tiene una desventaja significativa: con el transcurso del tiempo, los discos sufren fragmentación externa. Para ver cómo ocurre esto, examine la parte *b* de la figura anterior.

Acá se han eliminado dos archivos, *D* y *F*. Cuando se quita un archivo, sus bloques se liberan naturalmente, dejando una serie de bloques libres en el disco. El disco no se compacta al momento para quitar el hueco, ya que eso implicaría tener que copiar todos los bloques que van después del hueco, que podrían ser millones (una operación sumamente costosa en E/S). Como resultado, el disco al final consiste de archivos y huecos, como se ilustra en la figura.

Al principio esta fragmentación no es un problema, ya que cada nuevo archivo se puede escribir al final del disco, o después del anterior. Sin embargo, en un momento dado el disco se llenará y habrá que reutilizar el espacio libre de los huecos mediante algún tipo de algoritmo (ej: First-fit, Best-fit).

En última instancia podría ser necesario compactar el disco, lo cual es en extremo costoso.



Para reutilizar el espacio hay que mantener una lista de huecos. Sin embargo, cuando se cree un nuevo archivo será necesario conocer su tamaño final para poder elegir un hueco del tamaño correcto y colocarlo.

Imaginemos las consecuencias de tal diseño. El usuario empieza un editor de texto o procesador de palabras para poder escribir un documento. Lo primero que pide el programa es cuántos bytes tendrá el documento final. Esta pregunta se debe responder o el programa no continuará. Si el número dado finalmente es demasiado pequeño, el programa tiene que terminar prematuramente debido a que el hueco de disco está lleno y no hay lugar para colocar el resto del archivo.

Si el usuario trata de evitar este problema al proporcionar un número demasiado grande como tamaño final, por decir 100 MB, tal vez el editor no pueda encontrar un hueco tan grande y anuncie que el archivo no se puede crear. Desde luego que el usuario tiene la libertad de iniciar de nuevo el programa diciendo 50 MB esta vez y así en lo sucesivo hasta que se encuentre un hueco adecuado. Aun así, no es probable que este esquema haga que los usuarios estén felices.

Sin embargo, hay una situación en la que es factible la asignación contigua y, de hecho, se utiliza ampliamente: en los CD-ROMs. Acá todos los tamaños de los archivos se conocen de antemano y nunca cambiarán durante el uso subsiguiente del sistema de archivos del CD-ROM.

La historia se repite a menudo en las ciencias computacionales, a medida que van surgiendo nuevas generaciones de tecnología. En realidad, la asignación contigua se utilizó en los sistemas de archivos en discos magnéticos hace años, debido a su simpleza y alto rendimiento (la amabilidad con el usuario no contaba mucho entonces). Después se descartó la idea debido a la molestia de tener que especificar el tamaño final del archivo al momento de su creación. Pero con la llegada de los CD-ROMs, los DVDs y otros medios ópticos en los que se puede escribir sólo una vez, los archivos contiguos se volvieron, repentinamente, una buena idea otra vez.

Por lo tanto, es importante estudiar los antiguos sistemas y las ideas, que conceptualmente eran claras y simples, debido a que pueden aplicarse a los sistemas futuros en formas sorprendentes.

Asignación de lista enlazada (ligada).

El segundo método para almacenar archivos es mantener cada uno como una lista enlazada de bloques de disco, como se muestra en la figura siguiente.

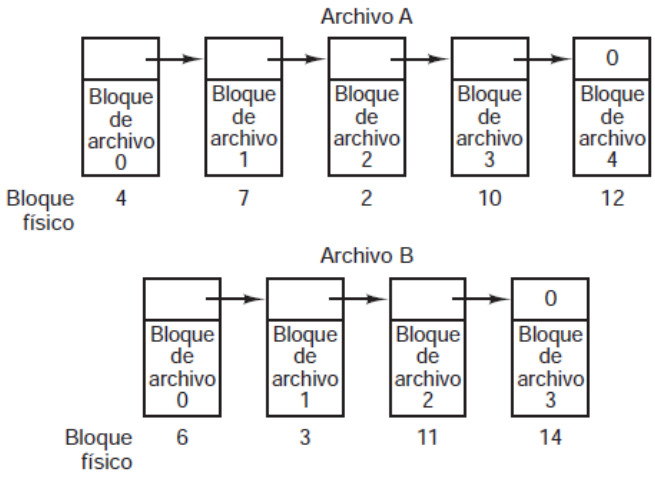


Ilustración 17 - Almacenamiento de un archivo como una lista enlazada de bloques de disco.

La primera palabra de cada bloque se utiliza como apuntador al siguiente. El resto del bloque es para los datos.

A diferencia de la asignación contigua, en este método se puede utilizar cada bloque del disco, por lo que se elimina la ***fragmentación externa*** (manteniéndose únicamente la fragmentación interna en el último bloque).

Además, para la entrada del directorio sólo le basta con almacenar la dirección de disco del primer bloque. El resto se puede encontrar a partir de ella.

Por otro lado, aunque la lectura secuencial de un archivo es directa, el acceso aleatorio es en extremo lento. Para llegar al bloque *n*, el sistema operativo tiene que empezar desde el principio y leer los *n – 1* bloques anteriores, uno a la vez. Es claro que tantas lecturas serán demasiado lentas.

Además, la cantidad de almacenamiento de datos en un bloque ya no es una potencia de dos, debido a que el apuntador ocupa unos cuantos bytes. Aunque no es fatal, tener un tamaño peculiar es menos eficiente debido a que muchos programas leen y escriben en bloques, cuyo tamaño es una potencia de dos.

Con los primeros bytes de cada bloque ocupados por un apuntador al siguiente bloque, leer el tamaño del bloque completo requiere adquirir y concatenar información de dos bloques de disco, lo cual genera un gasto adicional de procesamiento debido a la copia.

Ventaja Técnica	Desventaja Técnica
Cero fragmentación externa: Se aprovecha cada bloque libre del disco, sin importar dónde esté.	**Pérdida de Acceso Directo:** No se puede usar `lseek` de forma eficiente; hay que recorrer toda la cadena.
Crecimiento dinámico: El archivo puede expandirse constantemente mientras existan bloques libres.	**Gasto por Concatenación:** El tamaño de datos útil no es potencia de 2, ralentizando el procesamiento en memoria.

Asignación de lista enlazada utilizando una tabla en memoria (FAT).

Ambas desventajas de la asignación de lista enlazada se pueden eliminar si tomamos la palabra del apuntador de cada bloque de disco y la colocamos en una tabla en memoria.

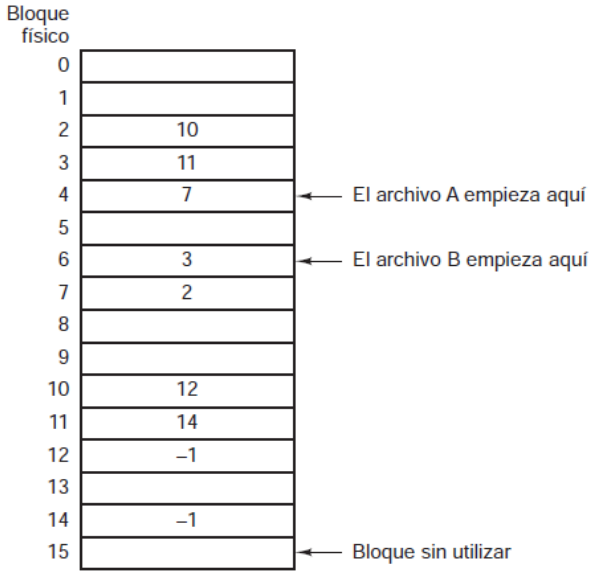


Ilustración 18 - Asignación de lista enlazada que utiliza una tabla de asignación de archivos en la memoria principal.

La figura anterior muestra cuál es la apariencia de la tabla para el ejemplo de la figura de la sección anterior (Asignación de lista enlazada).

En ambas figuras tenemos dos archivos. El archivo *A* utiliza los bloques de disco 4, 7, 2, 10 y 12, en ese orden y el archivo *B* utiliza los bloques de disco 6, 3, 11 y 14, en ese orden. Utilizando la tabla de la última figura, podemos empezar con el bloque 4 y seguir toda la cadena hasta el final. Lo mismo se puede hacer empezando con el bloque 6.

Ambas cadenas se terminan con un marcador especial (por ejemplo, -1) que no sea un número de bloque válido. Dicha tabla en memoria principal se conoce como ***FAT*** (*File Allocation Table,* Tabla de asignación de archivos).

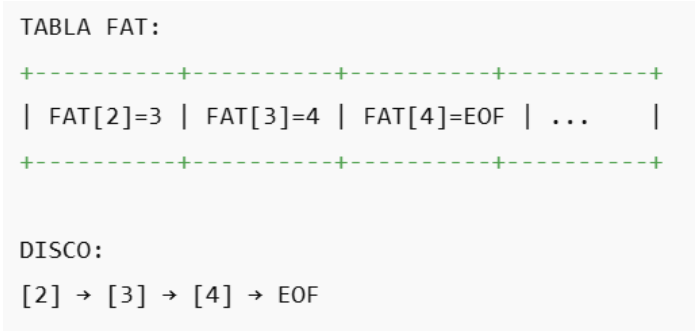


Ilustración 19 - El archivo empieza en el clúster 2. FAT dice que después del 2 viene el 3, después del 3 viene el 4, y el 4 es el fin del archivo (EOF).

Utilizando esta organización, el bloque completo está disponible para los datos. Además, el acceso aleatorio es mucho más sencillo. Aunque aún se debe seguir la cadena para encontrar un desplazamiento dado dentro del archivo, la cadena está completamente en memoria y se puede seguir sin necesidad de hacer referencias al disco. Al igual que el método anterior, la entrada de directorio necesita mantener sólo un entero (el número de bloque inicial) y aun así puede localizar todos los bloques, sin importar qué tan grande sea el archivo.

La principal desventaja de este método es que toda la tabla debe estar en memoria todo el tiempo para que funcione. Con un disco de 200 GB y un tamaño de bloque de 1 KB, la tabla necesita 200 millones de entradas, una para cada uno de los 200 millones de bloques de disco. Cada entrada debe tener un mínimo de 3 bytes. Para que la búsqueda sea rápida, deben tener 4 bytes. Así, la tabla ocupará 600 MB u 800 MB de memoria principal todo el tiempo, dependiendo de si el sistema está optimizado para espacio o tiempo. Esto no es muy práctico.

Es claro que la idea de la FAT no se escala muy bien en los discos grandes.

Ventajas (Pros)	Desventajas (Contras)
Bloques limpios: Los bloques miden una potencia de 2 exacta, acelerando el procesamiento en memoria.	**Uso de Memoria:** La tabla ocupa un espacio fijo y permanente en la memoria principal (RAM).
Saltos rápidos: El sistema busca las direcciones en la RAM, sin necesidad de leer el disco físico.	**Punto de fallo:** Si la tabla central se corrompe, se pierde el acceso a todo el disco.
Aprovecha el espacio: No deja huecos vacíos en el disco; usa cualquier bloque libre disponible.	**Límite de tamaño:** No escala bien en discos grandes y limita el tamaño máximo de cada archivo.

Asignación indexada (nodos-i).

Nuestro último método para llevar un registro de qué bloques pertenecen a cuál archivo es asociar con cada archivo una estructura de datos conocida como ***nodo-i*** (nodo-índice), la cual lista los atributos y las direcciones de disco de los bloques del archivo.

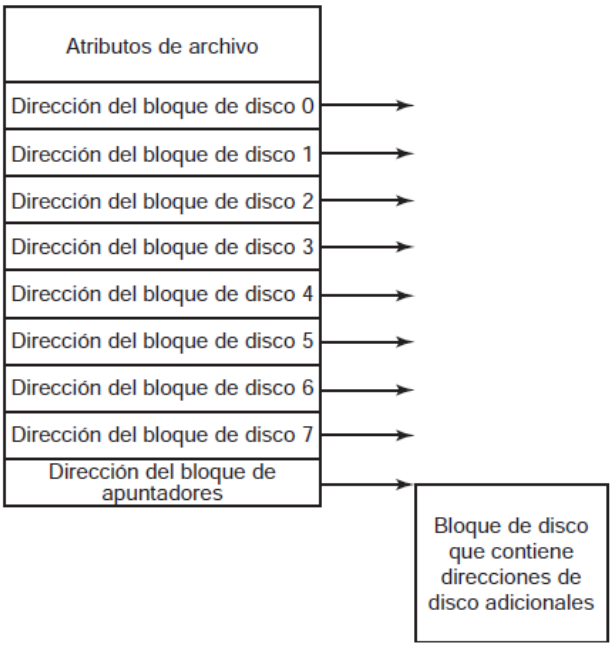


Ilustración 20 - Un nodo-i de ejemplo.

En la figura anterior se muestra un ejemplo simple. Dado el nodo-i, entonces es posible encontrar todos los bloques del archivo.

Cuando se formatea una partición con un sistema de archivos indexado (como ***Ext4*** en Linux), el sistema operativo reserva un sector fijo y específico del disco para crear una ***tabla de nodos-i***.

La gran ventaja de este esquema, en comparación con los archivos vinculados que utilizan una tabla en memoria, es que el nodo-i necesita estar en memoria sólo cuando está abierto el archivo correspondiente. Por lo general, el sistema operativo mantiene en la memoria principal una estructura dinámica llamada ***tabla de nodos-i activos***.

Criterio	Esquema FAT	Esquema Nodo-i (Asignación Indexada)
Enfoque	**Global:** Una única tabla para todo el disco.	**Individual:** Un bloque índice por cada archivo.
Permanencia en RAM	**Todo el tiempo:** Debe estar cargada por completo desde el inicio del sistema.	**Bajo demanda:** Se carga en memoria principal *únicamente* cuando el archivo se abre.

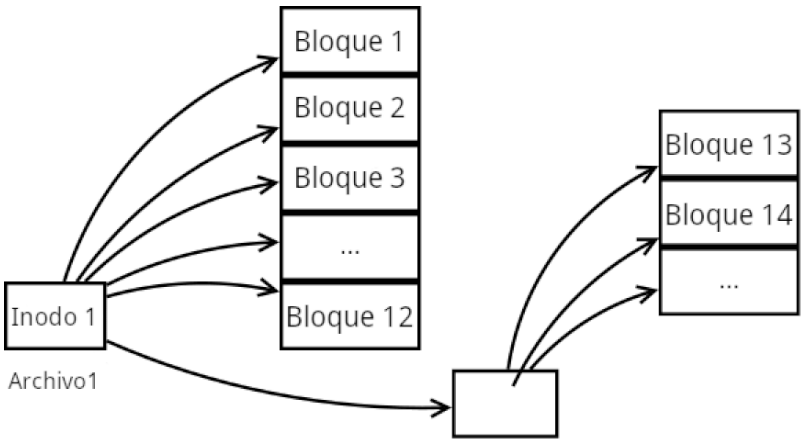
Si cada nodo-i ocupa *n* bytes y puede haber un máximo de *k* archivos abiertos a la vez, la memoria total ocupada por el arreglo que contiene los nodos-i para los archivos abiertos es de sólo *kn* bytes. Sólo hay que reservar este espacio por adelantado.

Por lo general, este arreglo es mucho más pequeño que el espacio ocupado por la tabla de archivos utilizando la técnica de listas enlazadas. La razón es simple: la tabla para contener la lista enlazada de todos los bloques de disco es proporcional en tamaño al disco en sí. Si el disco tiene *n* bloques, la tabla necesita *n* entradas. A medida que aumenta el tamaño de los discos, esta tabla aumenta linealmente con ellos.

En contraste, el esquema del nodo-i requiere un arreglo en memoria cuyo tamaño sea proporcional al número máximo de archivos que pueden estar abiertos a la vez. No importa si el disco es de 10 GB, de 100 GB o de 1000 GB.

Un problema con los nodos-i es que, si cada uno tiene espacio para un número fijo de direcciones de disco, será necesario preguntarse qué ocurre cuando un archivo crece más allá de este límite. Una solución es reservar la última dirección de disco no para un bloque de datos, sino para la dirección de un bloque que contenga más direcciones de bloques de disco.

Algo aún más avanzado sería que dos o más de esos bloques contuvieran direcciones de disco o incluso bloques de disco apuntando a otros bloques de disco llenos de direcciones.



Nodos-i en UNIX.

Todos los tipos de archivos UNIX se administran por el sistema operativo mediante los nodos-i. Como ya vimos, un nodo-i (nodo índice) es una estructura de control que contiene la información clave necesaria de un archivo particular para el sistema operativo. Varios nombres de archivos se pueden asociar con un único nodo-i, pero un nodo-i activo se asocia con exactamente un archivo, y cada archivo es controlado por exactamente un nodo-i.

Los atributos del archivo, así como sus permisos y otra información de control se almacenan en el nodo-i. La siguiente lista los atributos de archivos almacenados en el nodo-i de una implementación UNIX típica.

Modo de fichero	Espacio de 16 bits que almacena los permisos de acceso y ejecución asociados con el fichero.
	12-14 Tipo de fichero (regular, directorio, especial de caracteres o bloques, tubería FIFO)
	9-11 *Flags* de ejecución
	8 Permiso de lectura para el propietario
	7 Permiso de escritura para el propietario
	6 Permiso de ejecución para el propietario
	5 Permiso de lectura para el grupo del propietario
	4 Permiso de escritura para el grupo del propietario
	3 Permiso de ejecución para el grupo del propietario
	2 Permiso de lectura para el resto de usuarios
	1 Permiso de escritura para el resto de usuarios
	0 Permiso de ejecución para el resto de usuarios
Número de enlaces	Número de directorios que referencian a este nodo-i
Identificación del propietario	Propietario individual del fichero
Identificación de grupo del propietario	Grupo del propietario asociado con este fichero
Tamaño de fichero	Número de bytes del fichero
Direcciones de fichero	39 bytes de información de direcciones
Último acceso	Fecha del último acceso al fichero
Última modificación	Fecha de la última modificación del fichero
Nodo-i modificado	Fecha de la última modificación del nodo-i

Ilustración 21 - Información de un nodo-i UNIX residente en disco.

En el disco, hay una tabla de nodos-i, o lista de nodos-i, que contiene los nodos-i de todos los archivos del sistema de archivos. Cuando se abre un archivo, se trae su nodo-i a memoria principal y se almacena en una tabla de nodos-i residente en memoria.

Asignación en UNIX.

La asignación de archivos se realiza a nivel de bloque. La asignación es dinámica, es decir, cuando se necesita, en lugar de utilizar preasignación. Por tanto, los bloques de un archivo en disco no son necesariamente contiguos. Se utiliza un método indexado para guardar traza de cada archivo, con parte del índice almacenado en el nodo-i del archivo.

El nodo-i incluye 39 bytes de información de dirección que se organizan como trece direcciones de 3 bytes o punteros. Las primeras 10 direcciones apuntan a los primeros 10 bloques de datos del archivo. Si el archivo es mayor de 10 bloques, se utilizan uno o más niveles de indirección como se indica a continuación:

La dirección undécima en el nodo-i apunta a un bloque en disco que contiene la siguiente porción del índice. Esto se conoce como bloque indirecto simple. Este bloque contiene los punteros a siguientes bloques en el archivo.

Si el archivo contiene más bloques, la duodécima dirección del nodo-i apunta a un bloque indirecto doble. Este bloque contiene una lista de direcciones de bloques indirectos simples adicionales.

Cada uno de los bloques indirectos simples, a su vez, contiene punteros a bloques de archivos.

Si el archivo contiene todavía más bloques, la dirección decimotercera en el nodo-i apunta a un bloque indirecto triple que constituye un tercer nivel de indexación. Este bloque apunta a bloques indirectos dobles adicionales.

Todo esto se muestra en la figura siguiente.

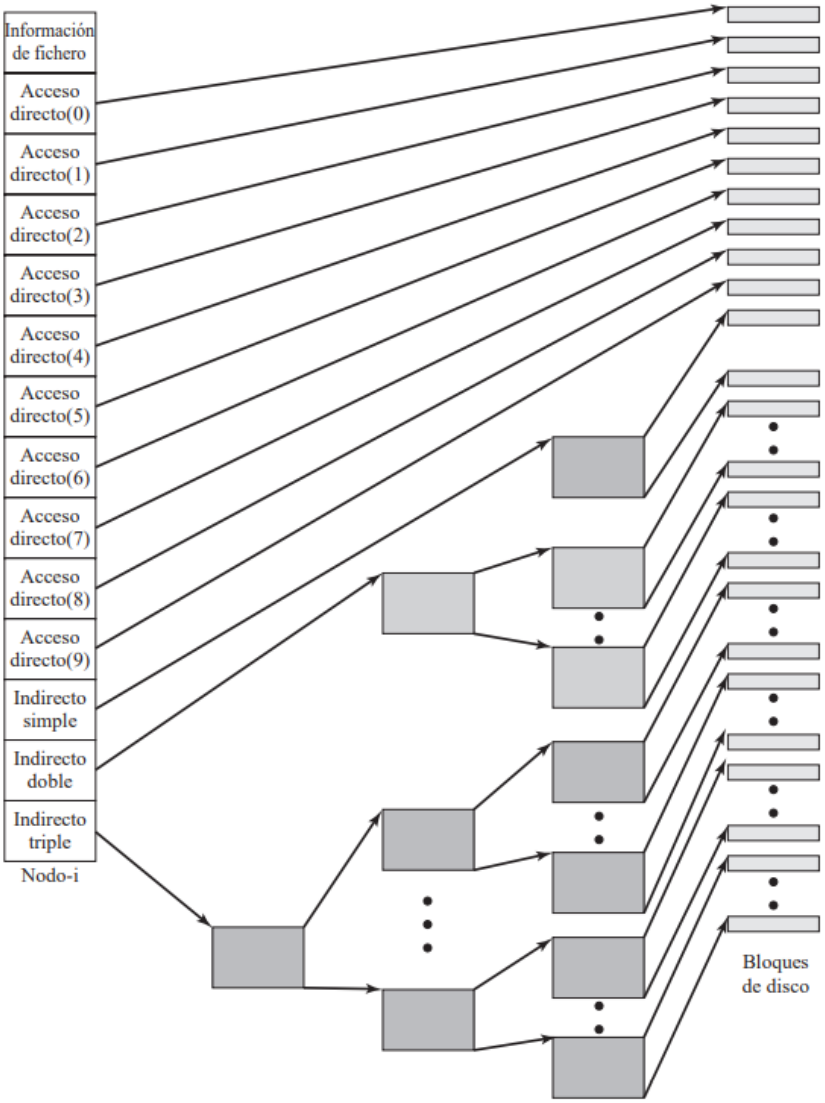


Ilustración 22 - Disposición de un archivo UNIX en un disco.

La primera entrada del nodo-i contiene información sobre este archivo. Las entradas restantes son las direcciones recién descritas.

El número total de bloques de datos en el archivo depende de la capacidad de los bloques de tamaño fijo en el sistema. En UNIX System V, la longitud de un bloque es 1 Kbyte y cada bloque puede contener un total de 256 direcciones de bloque. Por tanto, el tamaño máximo de un archivo con este esquema es cerca de 16 Gbytes.

Este esquema tiene varias ventajas:

1. El nodo-i es de tamaño fijo y relativamente pequeño y, por tanto, se puede almacenar en memoria principal durante periodos largos.
2. A los archivos más pequeños se puede acceder con poca o ninguna indirección, reduciendo el procesamiento y el tiempo de acceso a disco.

3. El tamaño máximo teórico de un archivo es suficientemente grande para satisfacer prácticamente todas las aplicaciones.

Referencias y bibliografía de la sección.

- Tanenbaum 4-3-2.
- Stallings pág. 574.

Organización de bloques y espacios libres.

Por lo general los archivos se almacenan en disco, así que la administración del espacio en disco es una cuestión importante para los diseñadores de sistemas de archivos.

Hay dos estrategias generales posibles para almacenar un archivo de *n* bytes: se asignan *n* bytes ***consecutivos*** de espacio en disco o el archivo se divide en varios ***bloques*** (no necesariamente) contiguos.

La misma concesión está presente en los sistemas de administración de memoria, entre la segmentación pura y la paginación.

Como hemos visto, almacenar un archivo como una secuencia contigua de bytes tiene el problema obvio de que, si un archivo crece, probablemente tendrá que moverse en el disco. El mismo problema se aplica a los segmentos en memoria, excepto que la operación de mover un segmento en memoria es rápida, en comparación con la operación de mover un archivo de una posición en el disco a otra.

Por esta razón, casi todos los sistemas de archivos dividen los archivos en bloques de tamaño fijo que no necesitan ser adyacentes.

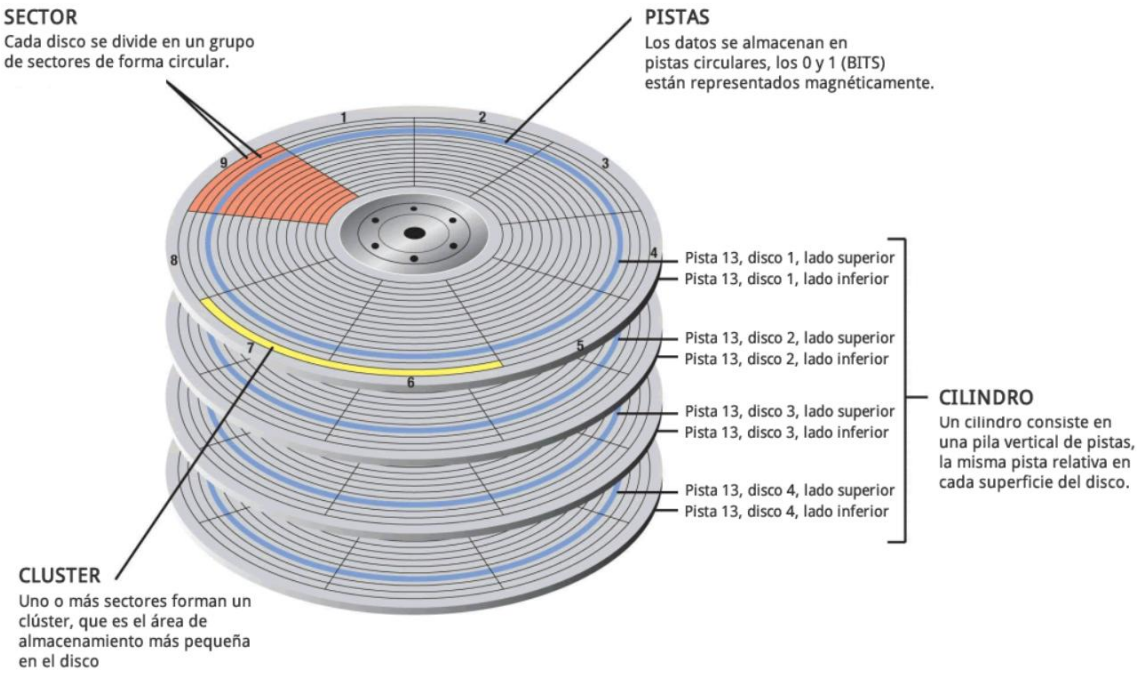
Puesto que el espacio del disco está limitado, necesitamos reutilizar el espacio de los archivos borrados para los nuevos archivos, siempre que sea posible. Para controlar el espacio libre del disco, el sistema mantiene una ***lista de espacio libre***.

La lista de espacio libre indica todos los bloques de disco libres, aquellos que no están asigna a ningún archivo o directorio. Para crear un archivo, exploramos la lista de espacio libre hasta obtener la cantidad de espacio requerida y asignamos ese espacio al nuevo archivo. A continuación, este espacio se elimina de la lista de espacio libre.

Cuando se borra un archivo, su espacio disco se añade a la lista de espacio libre. La lista de espacio libre, a pesar de su nombre, puede no estar implementada con una estructura de lista.

Tamaño de bloque.

Una vez que se ha decidido almacenar archivos en bloques de tamaño fijo, surge la pregunta acerca de qué tan grande debe ser el bloque. Dada la forma en que están organizados los discos, el sector, la pista y el cilindro son candidatos obvios para la unidad de asignación (aunque todos ellos son dependientes del dispositivo, lo cual es una desventaja). En un sistema de paginación, el tamaño de la página también es uno de los principales contendientes.



Tener un tamaño de bloque grande significa que cada archivo (incluso un archivo de 1 byte) ocupa un cilindro completo. También significa que los pequeños archivos desperdician una gran cantidad de espacio en disco. Por otro lado, un tamaño de bloque pequeño significa que la mayoría de los archivos abarcarán varios bloques y, por ende, necesitan varias búsquedas y retrasos rotacionales para leerlos, lo cual reduce el rendimiento. Por ende, si la unidad de asignación es demasiado grande, desperdiciamos espacio; si es demasiado pequeña, desperdiciamos tiempo.

Tamaño de Bloque	Ventaja	Desventaja
Grande (ej. un cilindro)	**Velocidad:** Lee mucha información de un solo golpe.	**Desperdicio:** Un archivo chico (de 1 byte) ocupa todo el bloque.
Pequeño (ej. 1 KB)	**Aprovechamiento:** No se pierde espacio con archivos chicos.	**Lentitud:** Los archivos grandes se fragmentan en miles de pedazos.

Registro de bloques libres.

Una vez que se ha elegido un tamaño de bloque, la siguiente cuestión es cómo llevar registro de los bloques libres. Hay dos métodos utilizados ampliamente, como se muestra en la figura siguiente.

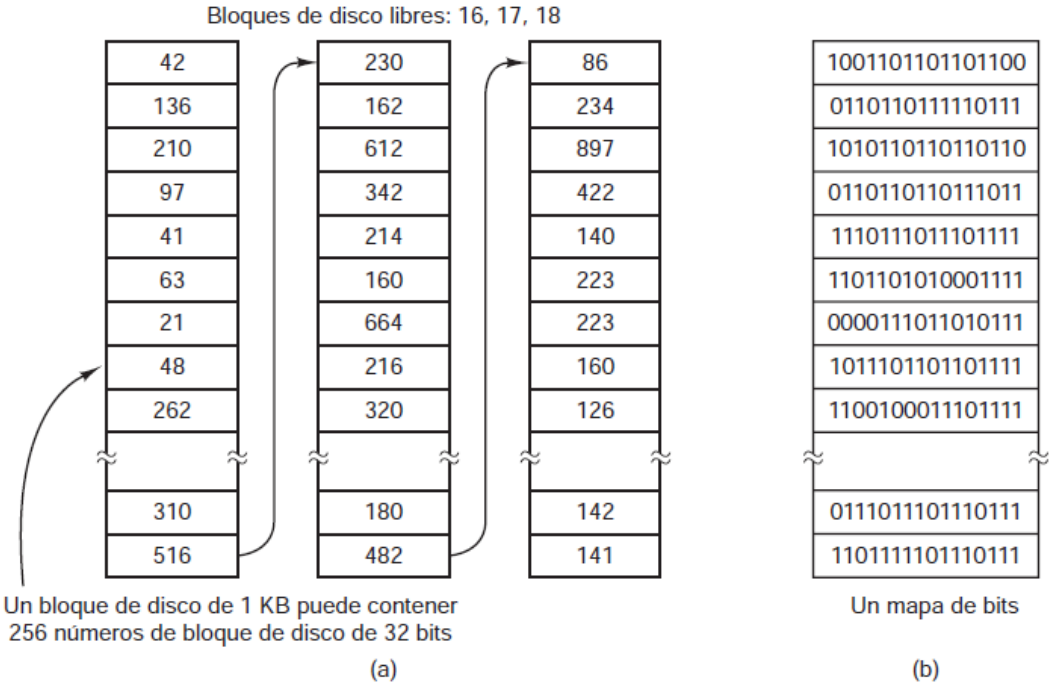
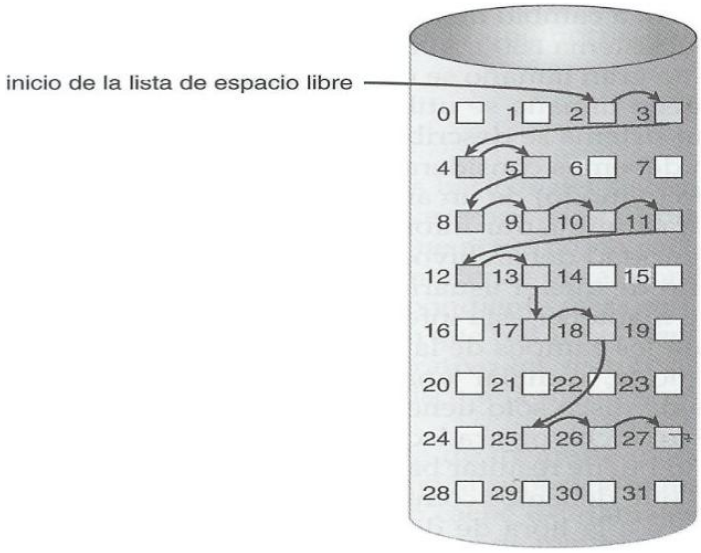


Ilustración 23 - (a) Almacenamiento de la lista de bloques libres en una lista enlazada. (b) Un mapa de bits.

Almacenamiento de la lista de bloques libres en una lista enlazada.

El primero consiste en utilizar la técnica de listas enlazadas que vimos en la sección anterior. La diferencia en este caso es que se trata de utilizar una ***lista enlazada de bloques libres de disco***, donde cada bloque contiene tantos números de bloques de disco ***libres*** como pueda. Con un bloque de 1 KB y un número de bloque de disco de 32 bits, cada bloque en la lista de bloques libres contiene los números de 255 bloques libres (se requiere una ranura para el apuntador al siguiente bloque).

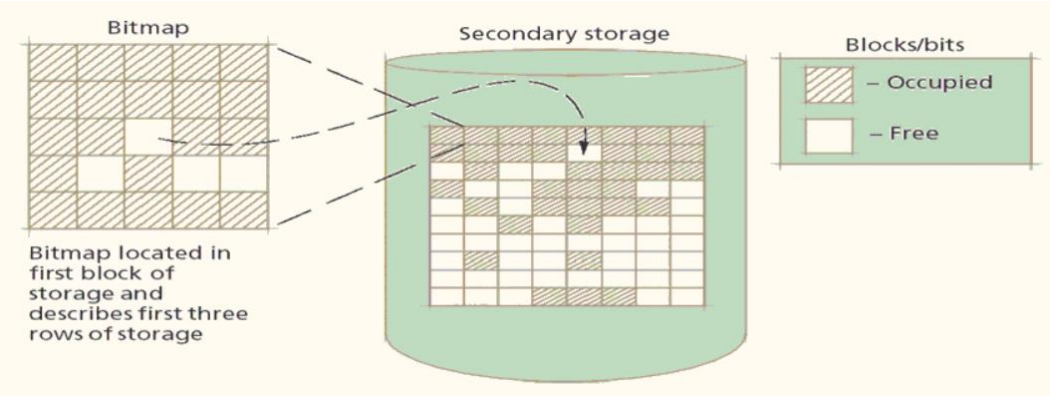


Consideremos un disco de 500 GB, que tiene aproximadamente 488 millones de bloques de disco. Para almacenar todas estas direcciones a 255 por bloque, se requiere una cantidad aproximada de 1.9 millones de bloques. En general se utilizan bloques libres para mantener la lista de bloques libres, por lo que en esencia el almacenamiento es gratuito.

Mapa de bits.

La otra técnica de administración del espacio libre es el mapa de bits. Un disco con *n* bloques requiere un mapa de bits con *n* bits. Los bloques libres se representan mediante 1s en el mapa, los bloques asignados mediante 0s (o viceversa). Para nuestro disco de 500 GB de ejemplo, necesitamos 488 millones de bits para el mapa, que requieren poco menos de 60,000 bloques de 1 KB para su almacenamiento.

No es sorpresa que el mapa de bits requiera menos espacio, ya que utiliza 1 bit por bloque, en comparación con 32 bits en el modelo de la lista enlazada. Sólo si el disco está casi lleno (es decir, que tenga pocos bloques libres) es cuando el esquema de la lista enlazada requiere menos bloques que el mapa de bits.



Si los bloques libres tienden a presentarse en series largas de bloques consecutivos, el sistema de la lista de bloques libres se puede modificar para llevar la cuenta de las series de bloques, en vez de bloques individuales. Una cuenta de 8, 16 o 32 bits se podría asociar con cada bloque dando el número de bloques libres consecutivos. En el mejor caso, un disco prácticamente vacío podría representarse mediante dos números: la dirección del primer bloque libre seguida de la cuenta de bloques libres.

Por otro lado, si el disco se fragmenta demasiado, es menos eficiente llevar el registro de las series que de los bloques individuales, debido a que no sólo se debe almacenar la dirección, sino también la cuenta.

Esta cuestión ilustra un problema con el que los diseñadores de sistemas operativos se enfrentan a menudo. Hay varias estructuras de datos y algoritmos que se pueden utilizar para resolver un problema, pero para elegir el mejor se requieren datos que los diseñadores no tienen y no tendrán sino hasta que el sistema se opere y se utilice con frecuencia. E incluso entonces, tal vez los datos no estén disponibles.

Con el método de la lista de bloques libres, sólo hay que mantener un bloque de apuntadores en la memoria principal. Al crear un archivo, los bloques necesarios se toman del bloque de apuntadores. Cuando este bloque se agota, se lee un nuevo bloque de apuntadores del disco. De manera similar, cuando se elimina un archivo se liberan sus bloques y se agregan al bloque de apuntadores en la memoria principal. Cuando este bloque se llena, se escribe en el disco.

Bajo ciertas circunstancias, este método produce operaciones de E/S de disco innecesarias.

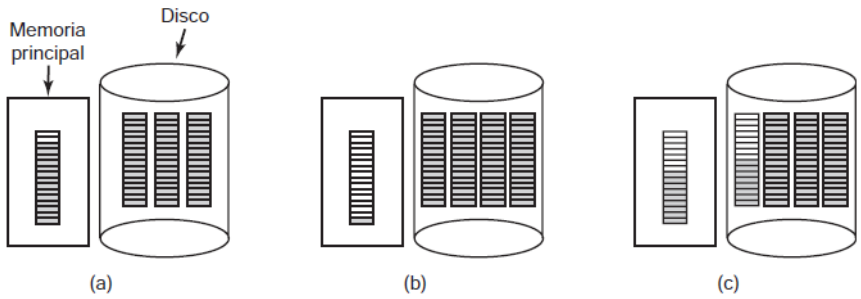


Ilustración 24 - (a) Un bloque casi lleno de apuntadoras a bloques de disco libres en la memoria y tres bloques de apuntadores en el disco. (b) Resultado de liberar un archivo de tres bloques. (c) Una estrategia alternativa para manejar los tres bloques libres. Las entradas sombreadas representan apuntadores a bloques de disco libres.

Consideremos la situación *a* de la figura anterior, donde el bloque de apuntadores en memoria tiene espacio para sólo dos entradas más. Si se libera un archivo de tres bloques, el bloque de apuntadores se desborda y tiene que escribirse en el disco, con lo cual se produce la situación de la parte *b*.

Si ahora se escribe un archivo de tres bloques, se tiene que volver a leer el bloque completo de apuntadores del disco, lo cual nos regresa la parte *a* de figura anterior. Si el archivo de tres bloques que se acaba de escribir era temporal, al liberarlo se necesitará otra escritura de disco para escribir el bloque completo de apuntadores de vuelta en el disco.

En resumen, cuando el bloque de apuntadores está casi vacío, una serie de archivos temporales de un tiempo corto de vida puede producir muchas operaciones de E/S de disco.

Un método alternativo que evita la mayor parte de estas operaciones de E/S de disco es dividir el bloque completo de apuntadores. Así, en vez de pasar de la parte *a* de la figura a la parte *b*, pasamos de *a* a *c* cuando se liberan tres bloques.

Ahora el sistema puede manejar una serie de archivos temporales sin realizar ninguna operación de E/S. Si el bloque en memoria se llena, se escribe en el disco y se lee el bloque medio lleno del disco. La idea aquí es mantener la mayor parte de los bloques de apuntadores en el disco llenos (para minimizar el uso del disco), pero mantener el que está en memoria lleno a la mitad, para que pueda manejar tanto la creación como la remoción de archivos sin necesidad de operaciones de E/S de disco en la lista de bloques libres.

Con un mapa de bits también es posible mantener sólo un bloque en memoria, usando el disco para obtener otro sólo cuando el primero se llena o se vacía. Un beneficio adicional de este método es que, al realizar toda la asignación de un solo bloque del mapa de bits, los bloques de disco estarán cerca uno del otro, con lo cual se minimiza el movimiento del brazo del disco.

Como el mapa de bits es una estructura de datos de tamaño fijo, si el kernel está paginado (parcialmente), el mapa de bits puede colocarse en memoria virtual y hacer que se paginen sus páginas según se requiera.

Cuotas de disco.

Para evitar que los usuarios ocupen demasiado espacio en disco, los sistemas operativos multiusuario proporcionan un mecanismo para imponer las cuotas de disco. La idea es que el administrador del sistema asigne a cada usuario una cantidad máxima de archivos y bloques y que el sistema operativo se asegure de que los usuarios no excedan sus cuotas.

Un mecanismo común se basa en localizar los atributos y las direcciones de disco y colocarlos en una tabla de archivos abiertos en la memoria principal, cuando un usuario abre un archivo.

Entre los atributos hay una entrada que indica quién es el propietario. Cualquier aumento en el tamaño del archivo se tomará de la cuota del propietario.

Una segunda tabla contiene el registro de cuotas para cada usuario con un archivo actualmente abierto, aun si el archivo fue abierto por alguien más. Esta tabla se muestra en la figura siguiente.

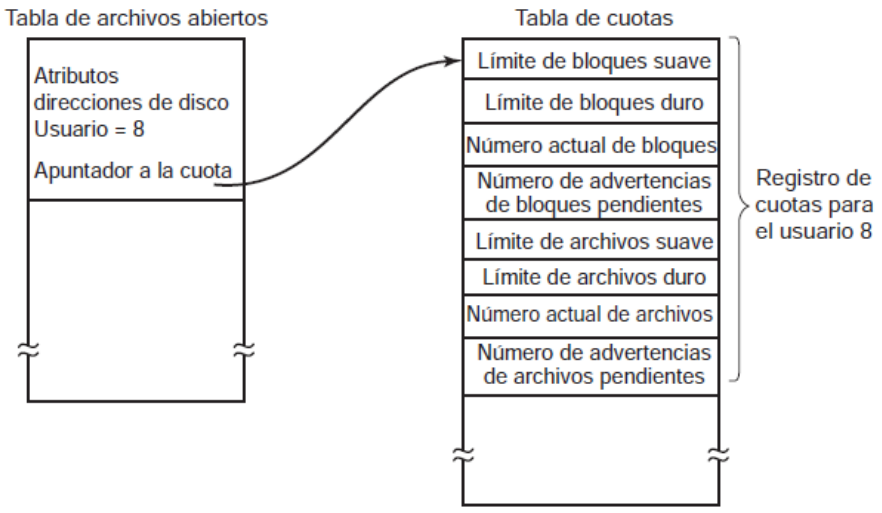


Ilustración 25 - El registro de cuotas se lleva en una tabla de cuotas, usuario por usuario.

Es un extracto de un archivo de cuotas en el disco para los usuarios cuyos archivos están actualmente abiertos. Cuando todos los archivos se cierran, el registro se escribe de vuelta en el archivo de cuotas.

Cuando se crea una nueva entrada en la tabla de archivos abiertos, se introduce en ella un apuntador al registro de cuotas del propietario, para facilitar la búsqueda de los diversos límites. Cada vez que se agrega un bloque a un archivo se incrementa el número total de bloques que se cargan al propietario y se realiza una verificación con los límites duro y suave.

Se puede exceder el ***límite suave***, pero el ***límite duro*** no. Un intento de agregar datos a un archivo cuando se ha llegado al límite de bloques duro producirá un error. También existen verificaciones similares para el número de archivos.

```
*** Report for user quotas on /dev/sda1 (/): ***
                        Block limits                File limits
User            used    soft    hard  grace    used  soft  hard  grace
----------------------------------------------------------------------
root      --   200000       0       0          1500     0      0
john      --    35000   40000   60000           600   4000   6000
jane      --    15000   20000   30000           300   2000   3000
```

Cuando un usuario trata de iniciar sesión, el sistema examina el archivo de cuotas para ver si el usuario ha excedido el límite suave para el número de archivos o el número de bloques de disco.

Si se ha violado uno de los límites, se muestra una advertencia y la cuenta de advertencias restantes se reduce en uno. Si la cuenta llega a cero en algún momento, el usuario ha ignorado

demasiadas veces la advertencia y no se le permite iniciar sesión. Para obtener permiso de iniciar sesión otra vez tendrá que hablar con el administrador del sistema.

Este método tiene la propiedad de que los usuarios pueden sobrepasar sus límites suaves durante una sesión, siempre y cuando eliminen el exceso cuando cierren su sesión. Los límites duros nunca se pueden exceder.

Agrupamiento.

Una alternativa a las técnicas de mapas de bits y listas enlazadas, consiste en almacenar las direcciones de n bloques libres en el primer bloque libre. Los primeros *n - 1* de estos bloques estarán realmente libres. El último bloque contendrá las direcciones de otros *n* bloques libres, etc. De este modo, podrán encontrarse rápidamente las direcciones de un gran número de bloques libres, a diferencia del caso en que se utilice la técnica estándar de lista enlazada.

Recuento.

Otra técnica consiste en aprovechar el hecho de que, generalmente, puede asignarse o liberarse simultáneamente varios bloques contiguos, particularmente cuando se asigna el espacio mediante el algoritmo de asignación contigua o mediante un mecanismo de agrupación en cluster[9]. Así, en lugar de mantener una lista de n direcciones de bloques de disco libres, podemos mantener la dirección del primer bloque libre y el número *n* de bloques libres contiguos que siguen a ese primer bloque. Cada entrada en la lista de espacio libre estará entonces compuesta por una dirección de disco y un contador. Aunque cada entrada requiere más espacio de lo que haría falta con una simple dirección de disco, la lista global será más corta, siempre y cuando el valor de ese contador sea generalmente superior a 1.

Referencias y bibliografía de la sección.

- Tanenbaum 4-4-1.
- Silberschatz 11-5.

Directorios.

Para llevar el registro de los archivos, los sistemas de archivos por lo general tienen ***directorios*** o ***carpetas***, que en muchos sistemas son también archivos.

[9] Un "cluster" en el contexto de los discos duros (también conocido como "unidad de asignación") es la unidad más pequeña de almacenamiento que el sistema operativo puede utilizar para almacenar datos.

Introducción a los directorios.

El directorio es una estructura que organiza y agrupa archivos dentro del sistema de almacenamiento, como un disco duro o una memoria USB.

El directorio puede considerarse como una tabla de símbolos que traduce los nombres de archivo a sus correspondientes entradas de directorio. Si adoptamos esta visión, es fácil comprender que el propio directorio puede organizarse de muchas formas. Queremos poder insertar entradas, borrar entradas, buscar una entrada conociendo su nombre y enumerar todas las entradas del directorio.

Cuando consideremos una estructura de directorio concreta, tendremos que tener presentes las operaciones que habrá que realizar con el directorio:

- **Búsqueda de un archivo.** Tenemos que poder explorar la estructura de directorio para encontrar la entrada correspondiente a un archivo concreto. Puesto que los archivos tienen nombres simbólicos y los nombres similares pueden indicar que existe una relación entre los archivos, también queremos poder encontrar todos los archivos cuyos nombres se correspondan con un patrón concreto.
- **Crear un archivo.** Es necesario poder crear nuevos archivos y añadirlos al directorio.
- **Borrar un archivo.** Cuando un archivo ya no es necesario, queremos poder eliminarlo del directorio.
- **Listar un directorio**. Tenemos que poder enumerar los archivos contenidos en un directorio y el contenido de la entrada de directorio correspondiente a cada uno de los archivos de la lista.
- **Renombrar un archivo.** Puesto que el nombre de un archivo representa su contenido para los usuarios, debemos poder cambiar el nombre cuando el contenido o el uso del archivo varíen. Renombrar un archivo puede también significar que se modifique su posición dentro de la estructura de directorio.
- **Recorrer el sistema de archivos.** Puede que queramos acceder a todos los directorios y a todos los archivos contenidos dentro de una estructura de directorios. Para conseguir una mayor fiabilidad, resulta conveniente guardar el contenido y la estructura de todo el sistema de archivos a intervalos regulares.

Sistemas de directorios de un solo nivel.

La forma más simple de un sistema de directorios es tener un directorio que contenga todos los archivos. Algunas veces se le llama ***directorio raíz***, pero como es el único, el nombre no importa mucho.

En las primeras computadoras personales, este sistema era común, en parte debido a que sólo había un usuario.

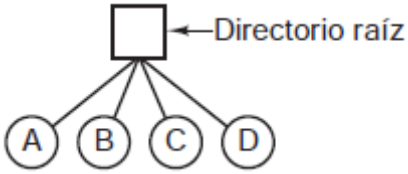


Ilustración 26 - Un sistema de directorio de un solo nivel que contiene cuatro archivos.

En la figura anterior se muestra un ejemplo de un sistema con un directorio. Acá el directorio contiene cuatro archivos. Las ventajas de este esquema son su simpleza y la habilidad de localizar archivos con rapidez; después de todo, sólo hay un lugar en dónde buscar.

A menudo se utilizaban en dispositivos simples como cámaras digitales y algunos reproductores de música portátiles.

Sistemas de directorios jerárquicos.

Tener un solo nivel es adecuado para aplicaciones dedicadas simples (e incluso se utilizaba en las primeras computadoras personales), pero para los usuarios modernos con miles de archivos, sería imposible encontrar algo si todos los archivos estuvieran en un solo directorio.

En consecuencia, se necesita una forma de agrupar los archivos relacionados. Por ejemplo, un profesor podría tener una colección de archivos que en conjunto formen un libro que está escribiendo para un curso, una segunda colección de archivos que contienen programas enviados por los estudiantes para otro curso, un tercer grupo de archivos que contenga el código de un sistema de escritura de compiladores avanzado que está construyendo, un cuarto grupo de archivos que contienen proposiciones de becas, así como otros archivos para correo electrónico, minutas de reuniones, artículos que está escribiendo, juegos, etcétera.

Lo que se necesita es una jerarquía (es decir, un árbol de directorios). Con este esquema, puede haber tantos directorios como se necesite para agrupar los archivos en formas naturales. Además, si varios usuarios comparten un servidor de archivos común, como se da el caso en muchas redes de empresas, cada usuario puede tener un directorio raíz privado para su propia jerarquía. Este esquema se muestra en la siguiente figura.

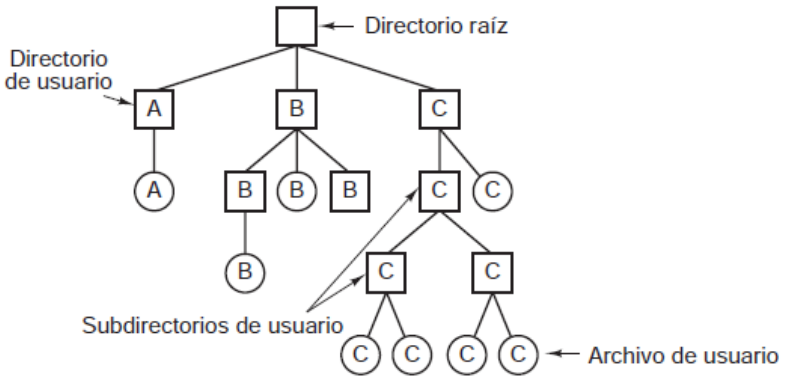


Ilustración 27 - Un sistema de directorios jerárquico.

Acá, cada uno de los directorios *A*, *B* y *C* contenidos en el directorio raíz pertenecen a un usuario distinto, dos de los cuales han creado subdirectorios para proyectos en los que están trabajando.

La capacidad de los usuarios para crear un número arbitrario de subdirectorios provee una poderosa herramienta de estructuración para que los usuarios organicen su trabajo. Por esta razón, casi todos los sistemas de archivos modernos se organizan de esta manera.

Modelo	Organización	Problema
Nivel Único (Plano)	Todos los archivos en una sola lista global.	**Colisión:** No permite duplicar nombres de archivos.
Jerárquico (Árbol)	Archivos agrupados en carpetas y subcarpetas.	**Recorrido:** El sistema debe buscar carpeta por carpeta.

Directorios en UNIX.

Los directorios se estructuran como un árbol jerárquico. Cada directorio contiene archivos y/u otros directorios. Un directorio que se encuentra dentro de otro directorio se denomina subdirectorio.
Como se mencionó anteriormente, un directorio es simplemente un archivo que contiene una lista de nombres de archivos más punteros a nodos-i asociados. La figura siguiente muestra la estructura global.

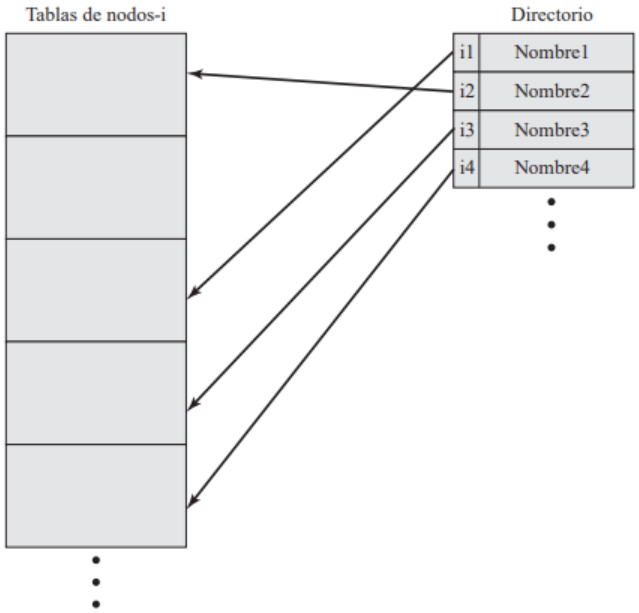


Ilustración 28 - Directorios y nodos-i UNIX.

Cada entrada de directorio (entradaD) contiene un nombre para el archivo asociado o subdirectorio más un entero llamado el número-i (número índice). Cuando se accede al archivo o directorio, su número-i se utiliza como un índice en la tabla de nodos-i.

Nombres de rutas.

Cuando el sistema de archivos está organizado como un árbol de directorios, se necesita cierta forma de especificar los nombres de los archivos. Por lo general se utilizan dos métodos distintos.

En el primer método, cada archivo recibe un ***nombre de ruta absoluto*** que consiste en la ruta desde el directorio raíz al archivo. Como ejemplo, la ruta */usr/ast/mailbox* significa que el directorio raíz contiene un subdirectorio llamado *usr*, que a su vez contiene un subdirectorio *ast*, el cual contiene el archivo *mailbox*. Los nombres de ruta absolutos siempre empiezan en el directorio raíz y son únicos.

En UNIX, los componentes de la ruta van separados por */*. En Windows el separador es \. En MULTICS era >.

Así, el mismo nombre de ruta se escribiría de la siguiente manera en estos tres sistemas:

- Windows \usr\ast\mailbox
- UNIX /usr/ast/mailbox
- MULTICS >usr>ast>mailbox

Sin importar cuál carácter se utilice, si el primer carácter del nombre de la ruta es el separador, entonces la ruta es ***absoluta***.

El otro tipo de nombre es el ***nombre de ruta relativa***. Éste se utiliza en conjunto con el concepto del ***directorio de trabajo*** (también llamado ***directorio actual***). Un usuario puede designar un directorio como el directorio de trabajo actual, en cuyo caso todos los nombres de las rutas que no empiecen en el directorio raíz se toman en forma relativa al directorio de trabajo. Por ejemplo, si el directorio de trabajo actual es */usr/ast*, entonces el archivo cuya ruta absoluta sea */usr/ast/mailbox* se puede referenciar simplemente como *mailbox*. En otras palabras, los siguientes comandos de UNIX hacen exactamente lo mismo si el directorio de trabajo es */usr/ast*:

```
cp /usr/ast/mailbox /usr/ast/mailbox.bak
```

```
cp mailbox mailbox.bak
```

A menudo es más conveniente la forma relativa, pero hace lo mismo que la forma absoluta.

Algunos programas necesitan acceder a un archivo específico sin importar cuál sea el directorio de trabajo. En ese caso, siempre deben utilizar nombres de rutas absolutas. Por ejemplo, un corrector ortográfico podría necesitar leer */usr/lib/dictionary* para realizar su trabajo. Debe utilizar el nombre de la ruta absoluta completo en este caso, ya que no sabe cuál será el directorio de trabajo cuando sea llamado. El nombre de la ruta absoluta siempre funcionará, sin importar cuál sea el directorio de trabajo.

Desde luego que, si el corrector ortográfico necesita un gran número de archivos de */usr/lib*, un esquema alternativo es que emita una llamada al sistema para cambiar su directorio de trabajo a */usr/lib* y que después utilice *dictionary* como el primer parámetro para open. Al cambiar en forma explícita el directorio de trabajo, sabe con certeza dónde se encuentra en el árbol de directorios, para poder entonces usar rutas relativas.

```
Programa en C
   ↓
Llamada a biblioteca (libc):
    chdir("/home/juan")
   ↓
↓ Transición al modo kernel ↓
   ↓
Llamada al sistema (syscall):
    sys_chdir("/home/juan")
   ↓
El kernel cambia el directorio
   ↓
Retorno al espacio de usuario
```

- `chdir()` → función de **biblioteca** (nivel usuario).
- `sys_chdir()` → **llamada al sistema** (nivel kernel).

Cada proceso tiene su propio directorio de trabajo, por lo que cuando éste cambia y después termina ningún otro proceso se ve afectado y no quedan rastros del cambio en el sistema de archivos.

De esta forma, siempre es perfectamente seguro para un proceso cambiar su directorio de trabajo cada vez que sea conveniente. Por otro lado, si un ***procedimiento de biblioteca*** cambia

el directorio de trabajo y no lo devuelve a su valor original cuando termina, el resto del programa tal vez no funcione, ya que el supuesto directorio de trabajo que debería tener ahora podría ser inválido.

Por esta razón, los procedimientos de biblioteca raras veces cambian el directorio de trabajo y cuando deben hacerlo siempre lo devuelven a su posición original antes de regresar.

```
Programa en C
   ↓
getcwd()   ← llamada a biblioteca (libc) para obtener directorio actual
   ↓
chdir("/tmp")   ← llamada a biblioteca para cambiar al nuevo directorio
   ↓
... operaciones en /tmp ...
   ↓
chdir(cwd)   ← llamada a biblioteca para volver al anterior
   ↓
Directorio restaurado
```

La mayoría de los sistemas operativos que proporcionan un sistema de directorios jerárquico tienen dos entradas especiales en cada directorio: “.” y “..”, que por lo general se pronuncian “punto” y “puntopunto”. Punto se refiere al directorio actual; puntopunto se refiere a su padre (excepto en el directorio raíz, donde se refiere a sí mismo).

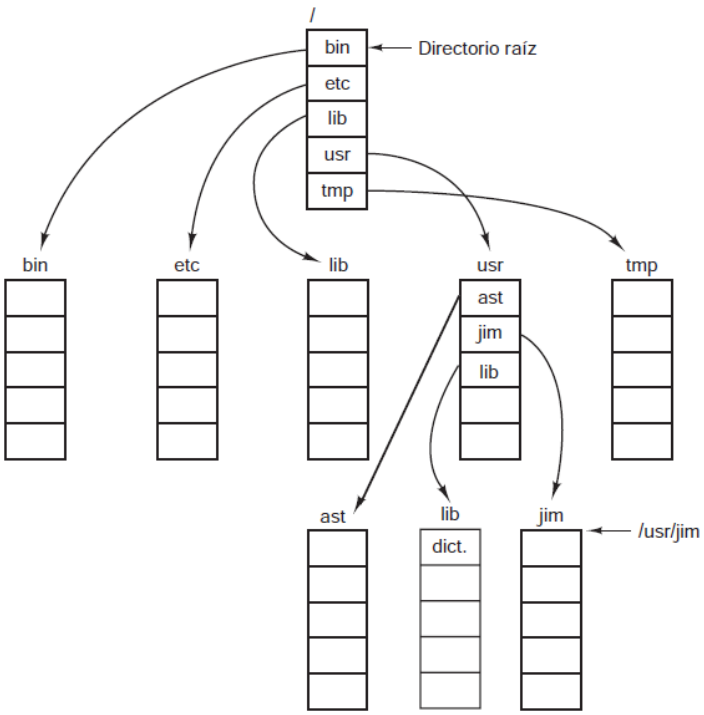


Ilustración 29 - Un árbol de directorios de UNIX.

Operaciones de directorios.

Las llamadas al sistema permitidas para administrar directorios exhiben más variación de un sistema a otro que las llamadas al sistema para los archivos. Para dar una impresión de lo que son y cómo funcionan, daremos un ejemplo (tomado de UNIX).

1. **Create**. Se crea un directorio. Está vacío, excepto por punto y puntopunto, que el sistema coloca ahí de manera automática (o en unos cuantos casos lo hace el programa ***mkdir***).

2. **Delete**. Se elimina un directorio. Se puede eliminar sólo un directorio vacío. Un directorio que sólo contiene a punto y puntopunto se considera vacío, ya que por lo general éstos no se pueden eliminar.

3. **Opendir**. Los directorios se pueden leer. Por ejemplo, para listar todos los archivos en un directorio, un programa de listado abre el directorio para leer los nombres de todos los archivos que contiene. Antes de poder leer un directorio se debe abrir, en forma análoga al proceso de abrir y leer un archivo.

4. **Closedir**. Cuando se ha leído un directorio, se debe cerrar para liberar espacio en la tabla interna.

5. **Readdir**. Esta llamada devuelve la siguiente entrada en un directorio abierto. Antes era posible leer directorios utilizando la llamada al sistema read común, pero ese método tiene la desventaja de forzar al programador a conocer y tratar con la estructura interna de los

directorios. En contraste, readdir siempre devuelve una entrada en formato estándar, sin importar cuál de las posibles estructuras de directorio se utilice.

6. **Rename**. En muchos aspectos, los directorios son sólo como archivos y se les puede cambiar el nombre de la misma forma que a los archivos.

7. **Link**. La vinculación (ligado) es una técnica que permite a un archivo aparecer en más de un directorio. Esta llamada al sistema especifica un archivo existente y el nombre de una ruta, creando un vínculo desde el archivo existente hasta el nombre especificado por la ruta. De esta forma, el mismo archivo puede aparecer en varios directorios. A un vínculo de este tipo, que incrementa el contador en el nodo-i del archivo (para llevar la cuenta del número de entradas en el directorio que contienen el archivo), se le llama algunas veces ***vínculo duro*** (o liga dura).

8. **Unlink**. Se elimina una entrada de directorio. Si el archivo que se va a desvincular sólo está presente en un directorio (el caso normal), se quita del sistema de archivos. Si está presente en varios directorios, se elimina sólo el nombre de ruta especificado. Los demás permanecen. En UNIX, la llamada al sistema para eliminar archivos (que vimos antes) es, de hecho, unlink.

La lista anterior contiene las llamadas más importantes, pero hay unas cuantas más; por ejemplo, para administrar la información de protección asociada con un directorio.

Una variante sobre la idea de vincular archivos es el ***vínculo simbólico*** (liga simbólica). En vez de tener dos nombres que apunten a la misma estructura de datos interna que representa un archivo, se puede crear un nombre que apunte a un pequeño archivo que nombre a otro. Cuando se utiliza el primer archivo, por ejemplo, abierto, el sistema de archivos sigue la ruta y busca el nombre al final. Después empieza el proceso de búsqueda otra vez, utilizando el nuevo nombre. Los vínculos simbólicos tienen la ventaja de que pueden traspasar los límites de los discos e incluso nombrar archivos en computadoras remotas. Sin embargo, su implementación es poco menos eficiente que los vínculos duros.

Referencias y bibliografía de la sección.

- Tanenbaum 4-2.
- Silberschatz 10-3.
- Stallings pág. 574.

Montaje.

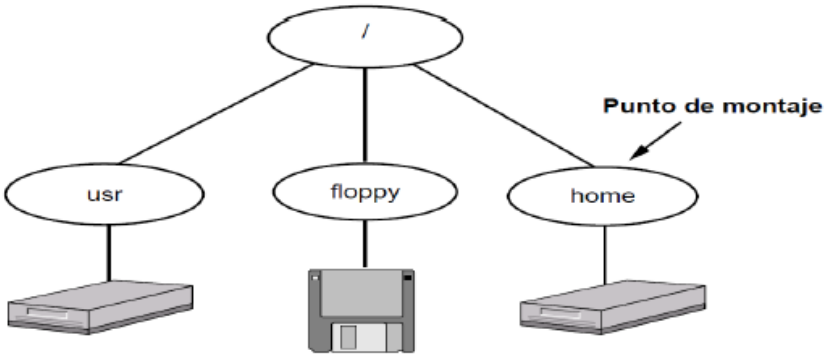
De la misma forma que un archivo debe abrirse antes de utilizarlo, un sistema de archivos debe ***montarse*** para poder estar disponible para los procesos del sistema. Más específicamente, la estructura de directorios puede estar formada por múltiples volúmenes,

que puede montarse para hacer que estén disponibles dentro del espacio de nombres del sistema de archivos.

El proceso de montaje es bastante simple. Al sistema operativo se le proporciona el ***nombre de dispositivo*** y el ***punto de montaje*** que es la ubicación dentro de la estructura de archivos a la que hay que conectar el sistema de archivos que se está montando.

Normalmente, el punto de montaje será un directorio vacío. Por ejemplo, en un sistema UNIX, un sistema de archivos que contenga los directorios principales de un usuario puede montarse como */home*; después, para acceder a la estructura de directorios contenida en ese sistema de archivos, podemos anteponer a los nombres de directorio el nombre */home*, como por ejemplo en */home/jane*.

Si montáramos ese sistema de archivos bajo el directorio */users* obtendríamos el nombre de ruta */users/jane*, que podríamos utilizar para acceder al mismo directorio.



A continuación, el sistema operativo verifica que el dispositivo contiene un sistema de archivos válido. Para ello, pide al controlador del dispositivo que lea el directorio de dispositivo y verifique que ese directorio tiene el formato esperado. Finalmente, el sistema operativo registra en su estructura de directorios que hay un sistema de archivo montado en el punto de montaje especificado.

Este esquema permite al sistema operativo recorrer su estructura de directorios, pasando de un sistema de archivos a otro según sea necesario.

Para ilustrar el montaje de archivos, consideremos el sistema de archivos mostrado en la figura siguiente:

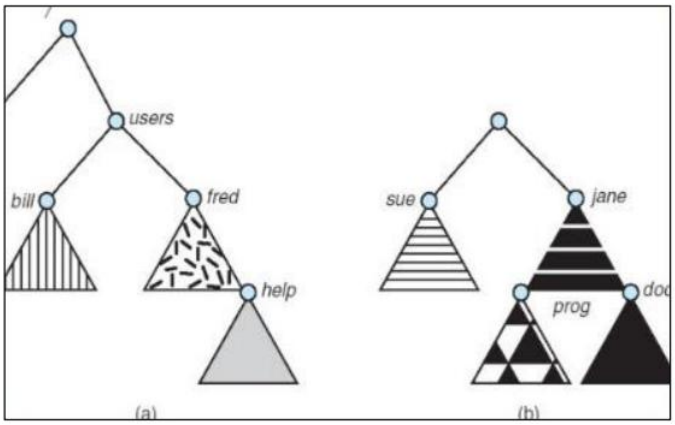


Ilustración 30 - Sistema de archivos. (a) Sistema existente. (b) Volumen no montado.

Los triángulos representan subárboles de directorios en los que estamos interesados. La parte *a* de la figura muestra un sistema de archivos existente, mientras que la parte *b* muestra un volumen no montado que reside en */device/dsk*.

En este punto, sólo puede accederse a los archivos del sistema de archivos existente. La figura siguiente muestra los efectos de mostrar en */users* el volumen que reside en */device/dsk*.

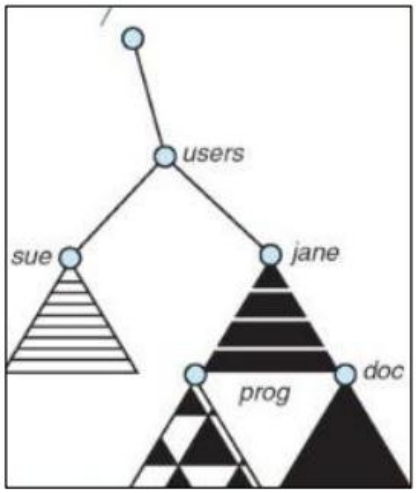


Ilustración 31 - Punto de montaje.

Los sistemas imponen una cierta semántica para clarificar la funcionalidad. Por ejemplo, un determinado sistema podría prohibir que se realizara un montaje en un directorio que contuviera archivos, o bien podría hacer que el sistema de archivos montado estuviera disponible en dichos directorios y ocultar los archivos existentes en dicho directorio hasta que el sistema de archivos se desmontara, en cuyo momento se prohíbe el uso de ese sistema de archivos y se permite el acceso a los archivos originales contenidos en el directorio.

Otro ejemplo, un determinado sistema podría permitir que se montara repetidamente el mismo sistema de archivos, en diferentes puntos de montaje; o, por el contrario, podría permitir un único montaje por cada sistema de archivos.

Consideremos el ejemplo del sistema operativo Macintosh. Cuando el sistema encuentra un disco por primera vez (los discos duros se localizan en el momento de iniciar el sistema) el sistema operativo Macintosh busca un sistema de archivos en el dispositivo. Si encuentra uno, monta automáticamente el sistema de archivos en el nivel raíz, añadiendo a la pantalla un icono de carpeta cuya etiqueta coincide con el nombre del sistema de archivos (tal como esté almacenado en el directorio del dispositivo). El usuario podrá entonces hacer clic sobre el icono y mostrar el sistema de archivos recién montado.

La familia Microsoft Windows de sistemas operativos mantiene una estructura ampliada de directorio en dos niveles, en la que a los dispositivos en los volúmenes se les asignan letras de unidad. Los volúmenes tienen una estructura de directorio en forma de grafo[10] general asociada con la letra de la unidad. Las versiones más recientes de Windows permiten montar un sistema de archivos en cualquier punto del árbol de directorios, al igual que en UNIX.

Los sistemas operativos Windows descubren automáticamente todos los dispositivos y montan todos los sistemas de archivo localizados en el momento de iniciar el sistema.

En algunos sistemas, como UNIX, los comandos de montaje son explícitos. Un determinado archivo de configuración del sistema contiene una lista de los dispositivos y puntos de montaje para realizar el montaje automático en el momento de iniciar el sistema, pero pueden ejecutarse otras operaciones de montaje manualmente.

Referencias y bibliografía de la sección.

- Silberschatz 10-4.

La bitácora.

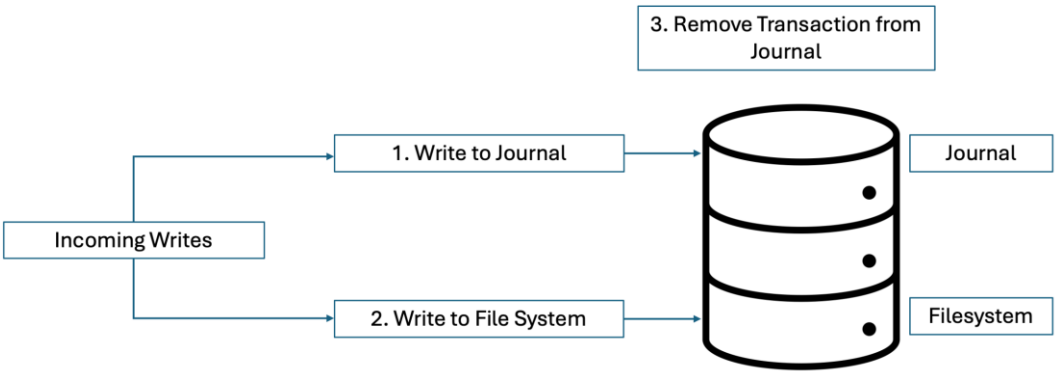
Como ya vimos, en un sistema de archivos ***estructurado por registro***, los archivos no se tratan como simples secuencias de bytes, sino como conjuntos de registros bien definidos (estructuras de datos con campos fijos o variables).

Aunque los sistemas de archivos estructurados por registro son una idea interesante, no se utilizan ampliamente, debido en parte a su alta incompatibilidad con los sistemas de archivos existentes.

Sin embargo, una de las ideas inherentes en ellos, la robustez frente a las fallas, se puede aplicar con facilidad a sistemas de archivos más convencionales. La idea básica es mantener un registro (bitácora o journal) de lo que va a realizar el sistema de archivos antes de hacerlo, por lo que, si el sistema falla antes de poder realizar su trabajo planeado, al momento de re-arrancar el sistema puede buscar en el registro para ver lo que estaba ocurriendo al momento

[10] Un grafo (también conocido como graph en inglés) es una estructura de datos abstracta que se utiliza para representar relaciones entre objetos. Consiste en un conjunto de nodos (también llamados vértices) y un conjunto de aristas (o conexiones) que establecen relaciones entre los nodos.

de la falla y terminar el trabajo. Dichos sistemas de archivos, conocidos como ***sistemas de archivos por bitácora*** (*Journaling files system,* ***JFS***), se encuentran en uso actualmente.



El sistema de archivos ***NTFS*** de Microsoft, así como los sistemas ***ext3*** y ***ReiserFS*** de Linux son todos por bitácora.

Sistema	Plataforma	Tipo de journaling
ext3 / ext4	Linux	Datos y metadatos
NTFS	Windows	Metadatos
XFS, JFS, ReiserFS	Linux / UNIX	Datos y metadatos

Para ver la naturaleza del problema, consideremos una operación simple que ocurre todo el tiempo: eliminar un archivo.

Esta operación (en UNIX) requiere tres pasos:

1. Quitar el archivo de su directorio.
2. Liberar el nodo-i y pasarlo a la reserva de nodos-i libres.
3. Devolver todos los bloques de disco a la reserva de bloques de disco libres.

En Windows se requieren pasos similares. En la ausencia de fallas del sistema, el orden en el que se realizan estos pasos no importa; en la presencia de fallas, sí.

Supongamos que se completa el primer paso y después el sistema falla. El nodo-i y los bloques de archivo no estarán accesibles desde ningún archivo, pero tampoco estarán disponibles para ser reasignados; sólo se encuentran en alguna parte del limbo, disminuyendo los recursos disponibles. Si la falla ocurre después del siguiente paso, sólo se pierden los bloques.

Si el orden de las operaciones se cambia y el nodo-i se libera primero, entonces después de re-arrancar, el nodo-i se puede reasignar, pero la entrada de directorio anterior seguirá apuntando a él y por ende al archivo incorrecto.

Si los bloques se liberan primero, entonces una falla antes de limpiar el nodo-i indicará que una entrada de directorio válida apunta a un nodo-i que lista los bloques que ahora se encuentran en la reserva de almacenamiento libre y que probablemente se reutilicen en breve, produciendo dos o más archivos que compartan al azar los mismos bloques. Ninguno de estos resultados es bueno.

Lo que hace el sistema de archivos por bitácora es escribir primero una entrada de registro que liste las tres acciones a completar. Después la entrada de registro se escribe en el disco (y como buena medida, posiblemente se lea otra vez del disco para verificar su integridad). Sólo hasta que se ha escrito la entrada de registro es cuando empiezan las diversas operaciones.

Una vez que las operaciones se completan con éxito, se borra la entrada de registro. Si ahora el sistema falla, al momento de recuperarse el sistema de archivos puede verificar el registro para ver si había operaciones pendientes. De ser así, todas ellas se pueden volver a ejecutar (múltiples veces, en caso de fallas repetidas) hasta que el archivo se elimine en forma correcta.

Para que funcione el sistema por bitácora, las operaciones registradas deben ser ***idempotentes***[11], lo cual significa que pueden repetirse todas las veces que sea necesario sin peligro. Las operaciones como "Actualizar el mapa de bits para marcar el nodo-i *k* o el bloque *n* como libre" se pueden repetir hasta que las todas las operaciones se completen sin peligro. De manera similar, las operaciones de buscar en un directorio y eliminar una entrada cualquiera también son idempotentes.

Por otro lado, la operación de agregar los bloques recién liberados del nodo-i *K* al final de la lista libre no es idempotente, debido a que éstos tal vez ya se encuentren ahí. La operación más costosa "Buscar en la lista de bloques libres y agregarle el bloque *n* si no está ya presente", también es idempotente.

[11] La idempotencia se refiere a la propiedad de una operación o función de producir el mismo resultado independientemente de cuántas veces se aplique o se ejecute.

```
Operación normal:
1. Escribir archivo X
2. Registrar en bitácora 📝
3. Aplicar cambio en disco
4. Marcar operación como completada ✅

↓ Ocurre un fallo (corte de energía) justo después del paso 2

Reinicio del sistema:
1. Revisar bitácora
2. Detectar operación incompleta
3. Rehacer o revertir cambios
4. Limpiar bitácora
```

Los sistemas de archivos por bitácora tienen que organizar sus estructuras de datos y operaciones que pueden registrarse, de manera que todas ellas sean idempotentes. Bajo estas condiciones, la recuperación de errores puede ser rápida y segura.

Para una mayor confiabilidad, un sistema de archivos puede introducir el concepto de las bases de datos conocido como ***transacción atómica*.** Cuando se utiliza este concepto, varias acciones se pueden agrupar mediante las operaciones ***begin transaction*** y ***end transaction***. Así, el sistema de archivos sabe que debe completar todas las operaciones agrupadas o ninguna de ellas, pero ninguna otra combinación.

En otras palabras, los marcadores begin transaction y end transaction (o equivalentes) delimitan los cambios que deben aplicarse como una unidad atómica: o se hacen todos, o no se hace ninguno.

Etapa	Descripción
BEGIN TRANSACTION	Indica el inicio de un grupo de operaciones que deben tratarse como una sola transacción. Se comienzan a registrar los cambios en la **bitácora (journal)**.
Operaciones	Se escriben en el log las modificaciones planeadas: escribir bloques, crear archivos, cambiar metadatos, etc.
END TRANSACTION (COMMIT)	Marca que todas las operaciones se registraron correctamente en la bitácora. Solo después de este punto, el sistema puede aplicar los cambios al disco principal.

NTFS tiene un sistema por bitácora extenso y su estructura rara vez se corrompe debido a las fallas en el sistema. Ha estado en desarrollo desde la primera vez que se liberó con Windows NT en 1993.

El primer sistema de archivos por bitácora de Linux fue ReiserFS, pero su popularidad se vio impedida por el hecho de que era incompatible con el sistema de archivos ext2, que entonces

era el estándar. En contraste, ext3, que es un proyecto menos ambicioso que ReiserFS, también se hace por bitácora, al tiempo que mantiene la compatibilidad con el sistema ext2 anterior.

Referencias y bibliografía de la sección.

- Tanenbaum 4-3-6.

Sistemas compartidos.

En un sistema multiusuario, existe casi siempre el requisito de permitir que los archivos se compartan entre varios usuarios. Existen dos aspectos relacionados: derechos de acceso y la gestión de acceso simultáneo.

Derechos de acceso.

El sistema de archivos debería proporcionar una herramienta flexible para permitir la compartición de archivos extensiva entre los usuarios. El sistema de archivos debería proporcionar varias opciones de tal forma que el acceso a un archivo particular se pueda controlar.

Típicamente, a los usuarios o grupos de usuarios se les concede ciertos derechos de acceso a un archivo. Se ha utilizado un amplio rango de derechos de acceso. La siguiente lista es representativa de los derechos de acceso que se asignan a un usuario particular para un determinado archivo:

- **Ninguno.** El usuario no puede incluso conocer la existencia del archivo, y por tanto, tampoco puede acceder a él. Para forzar esta restricción, el usuario no tiene permiso de lectura del directorio en el cual se incluye este archivo.
- **Conocimiento**. El usuario puede determinar si el archivo existe y quién es su propietario. El usuario entonces es capaz de solicitar al propietario derechos de acceso adicionales.
- **Ejecución**. El usuario puede cargar y ejecutar un programa, pero no copiarlo. Los programas propietarios utilizan normalmente estas restricciones.
- **Lectura**. El usuario puede leer el archivo para cualquier propósito, incluyendo copia y ejecución. Algunos sistemas son capaces de forzar una distinción entre ver y copiar. En el primer caso, el usuario puede ver el contenido del archivo, pero no puede realizar una copia.
- **Adición**. El usuario puede añadir datos al archivo, frecuentemente sólo al final, pero no puede modificar o borrar cualquiera de los contenidos del archivo. Este derecho es útil para recolectar datos de varias fuentes.
- **Actualización**. El usuario puede modificar, borrar o añadir datos al archivo. Esto normalmente incluye escribir el archivo al inicio, reescribirlo completa o parcialmente

y borrar todos o una porción de los datos. Algunos sistemas diferencian entre distintos grados de actualización.

- **Cambio de protección.** El usuario puede cambiar los derechos de acceso otorgados a otros usuarios. Normalmente, sólo el propietario del archivo goza de este derecho. En algunos sistemas, el propietario puede extender este derecho a otros. Para prevenir del abuso de este mecanismo, el propietario del archivo normalmente puede especificar qué derechos podría cambiar un usuario con este tipo de permiso.
- **Borrado.** El usuario puede borrar el archivo del sistema de archivos.

Estos derechos se pueden considerar como una jerarquía, con cada uno de los derechos conteniendo a aquellos que le preceden. Por tanto, si un usuario particular tiene el permiso de actualización para un archivo concreto, dicho usuario también tendrá los siguientes derechos: conocimiento, ejecución, lectura y adición.

A un usuario se le considera ***propietario*** de un determinado archivo, normalmente a la persona que inicialmente creó un archivo.

El propietario tiene todos los derechos de acceso listados previamente y puede conceder permisos a otros usuarios. Se pueden proporcionar diferentes accesos a distintas clases de usuarios:

- **Usuario específico.** Usuarios individuales que se designan por el identificador del usuario.
- **Grupos de usuarios.** Un conjunto de usuarios que no se definen individualmente. El sistema debe tener alguna forma de gestionar la membresía de los grupos de usuarios.
- **Todos.** Todos los usuarios que tienen acceso a este sistema. Dichos archivos se consideran archivos públicos.

Cuando un sistema operativo tiene múltiples usuarios, las cuestiones relativas a la compartición de archivos, a la denominación de archivos y a la protección de archivos cobran una gran importancia.

Dada una estructura de directorio que permite que los usuarios compartan archivos, el sistema debe adoptar un papel de mediador en lo que a la compartición de archivos respecta. El sistema puede permitir que un usuario acceda a los archivos de otros usuarios de manera predeterminada, o puede por el contrario requerir que los usuarios concedan explícitamente el acceso a sus archivos. Estos son los problemas del control de acceso y de la protección.

Para implementar la compartición y los mecanismos de protección, el sistema debe mantener más atributos de los archivos y de los directorios que los que se necesitan en un sistema monousuario.

Aunque históricamente se han adoptado diversos enfoques para satisfacer estos requisitos, la mayoría de los sistemas han terminado por evolucionar para usar el concepto de ***propietario*** (o usuario) de un archivo o directorio y el concepto de ***grupo***.

Como dijimos, el propietario es el usuario que puede cambiar los atributos y conceder el acceso y que dispone del máximo grado de control sobre e archivo.

El atributo de grupo define un subconjunto de usuarios que pueden compartir el acceso al archivo.

Por ejemplo, el propietario de un archivo en un sistema UNIX puede ejecutar todas las operaciones sobre el archivo, mientras que los miembros del grupo de un archivo sólo pueden ejecutar un subconjunto de esas operaciones, y todos los restantes usuarios podrán ejecutar otro subconjunto de operaciones.

El propietario del archivo es quien define exactamente qué operaciones pueden ser ejecutadas por los miembros del grupo y por los restantes usuarios.

Los identificadores del propietario y del grupo de un archivo (o directorio) determinado se almacenan junto con los otros atributos del archivo. Cuando un usuario solicita realizar una operación sobre un archivo, se puede comparar el ID del usuario con el atributo de propietario para determinar si el usuario solicitante es el propietario del archivo. De la misma manera, pueden compararse los identificadores de grupo. El resultado indicará qué permisos son aplicables. A continuación, el sistema aplicará dichos permisos a la operación solicitada y la autorizará o denegará.

Muchos sistemas disponen de múltiples sistemas de archivos locales, incluyendo volúmenes compuestos por un único disco o múltiples volúmenes almacenados en múltiples discos conectados. En estos casos, los procesos de comprobación de las identidades y de comprobación de los permisos son bastante sencillos, una vez que los sistemas de archivos hayan sido montados.

Acceso simultaneo.

Cuando se garantiza acceso de adición o actualización de un archivo a más de un usuario, el sistema operativo o sistema de gestión de archivos debe forzar una disciplina.

Una técnica de fuerza bruta consiste en permitir al usuario bloquear el archivo completo cuando se va a actualizar.

Un control de grano más fino implica el bloqueo de registros individuales durante la actualización.

Esencialmente, este es el problema de los ***lectores/escritores***. Se deben tratar aspectos de ***exclusión mutua*** e ***interbloqueos*** a la hora de diseñar capacidades de acceso compartidas.

Archivos compartidos.

Cuando hay varios usuarios trabajando en conjunto en un proyecto, a menudo necesitan compartir archivos. Como resultado, con frecuencia es conveniente que aparezca un archivo compartido en forma simultánea en distintos directorios que pertenezcan a distintos usuarios.

La figura siguiente muestra el sistema de archivos.

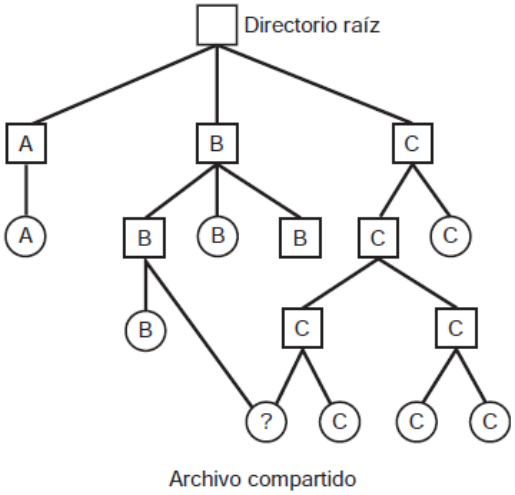


Ilustración 32 - Sistema de archivos que contiene un archivo compartido.

La conexión entre el directorio de *B* y el archivo compartido se conoce como un **vínculo** (liga). El sistema de archivos en sí es ahora un **Gráfico acíclico dirigido** (*Directed Acyclic Graph,* **DAG**) en vez de un árbol.

Compartir archivos es conveniente, pero también introduce ciertos problemas. Para empezar, si los directorios en realidad contienen direcciones de disco, entonces habrá que realizar una copia de las direcciones de disco en el directorio de *B* cuando se ligue el archivo. Si *B* o *C* agregan posteriormente al archivo, los nuevos bloques se listarán sólo en el directorio del usuario que agregó los datos. Los cambios no estarán visibles para el otro usuario, con lo cual fracasa el propósito de la compartición.

Este problema se puede resolver de dos formas. En la primera solución, los bloques de disco no se listan en los directorios, sino en una pequeña estructura de datos asociada con el archivo en sí. Entonces, los directorios apuntarían sólo a la pequeña estructura de datos. Éste es el esquema que se utiliza en UNIX (donde la pequeña estructura de datos es el nodo-i).

En la segunda solución, *B* se vincula a uno de los archivos de *C* haciendo que el sistema cree un archivo, de tipo LINK e introduciendo ese archivo en el directorio de *B*. El nuevo archivo contiene sólo el nombre de la ruta del archivo al cual está vinculado. Cuando *B* lee del archivo vinculado, el sistema operativo ve que el archivo del que se están leyendo datos es de tipo LINK, busca el nombre del archivo y lee el archivo. A este esquema se le conoce como **vínculo simbólico (liga simbólica)**, para contrastarlo con el tradicional vínculo (duro).

Cada uno de estos métodos tiene sus desventajas. En el primer método, al momento en que *B* se vincula al archivo compartido, el nodo-i registra al propietario del archivo como *C*. Al crear un vínculo no se cambia la propiedad, sino incrementa la cuenta de vínculos en el nodo-i, por lo que el sistema sabe cuántas entradas de directorio actualmente apuntan al archivo.

Si *C* posteriormente trata de eliminar el archivo, el sistema se enfrenta a un problema. Si elimina el archivo y limpia el nodo-i, *B* tendrá una entrada de directorio que apunte a un nodo-i inválido.

Si el nodo-i se reasigna más tarde a otro archivo, el vínculo de *B* apuntará al archivo incorrecto.

El sistema puede ver de la cuenta en el nodo-i que el archivo sigue todavía en uso, pero no hay una manera sencilla de que encuentre todas las entradas de directorio para el archivo, para que pueda borrarlas. Los apuntadores a los directorios no se pueden almacenar en el nodo-i, debido a que puede haber un número ilimitado de directorios.

Lo único por hacer es eliminar la entrada de directorio de *C*, pero dejar el nodo-i intacto, con la cuenta establecida en 1, como se muestra en la parte *c* de la figura siguiente.

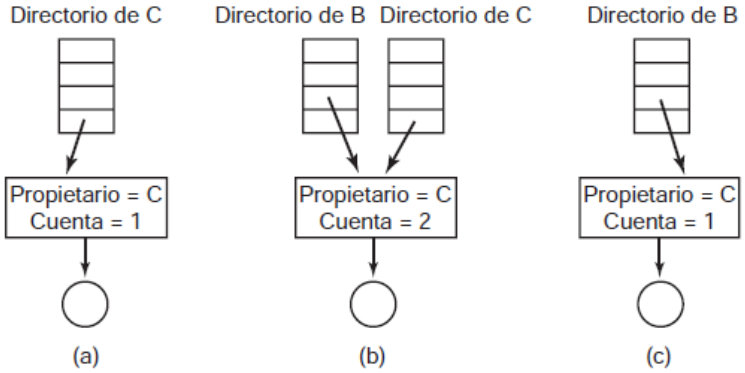


Ilustración 33 - (a) Situación previa a la vinculación. (b) Después de crear el vínculo. (c) Después de que el propietario original elimina el archivo.

Ahora tenemos una situación en la que *B* es el único usuario que tiene una entrada de directorio para un archivo que pertenece a *C*. Si el sistema realiza la contabilidad o tiene cuotas, seguirá cobrando a *C* por el archivo hasta que *B* decida eliminarlo, si acaso lo hace, momento en el cual la cuenta será 0 y el archivo se eliminará.

Con los vínculos simbólicos no se produce este problema, debido a que sólo el verdadero propietario tiene un apuntador al nodo-i. Los usuarios que han creado vínculos al archivo sólo tienen nombres de ruta, no apuntadoras a nodos-i. Cuando el *propietario* elimina el archivo, éste se destruye.

Los intentos posteriores por utilizar el archivo vía un vínculo simbólico fallarán cuando el sistema no pueda localizar el archivo. Al eliminar un vínculo simbólico, el archivo no se ve afectado.

El problema con los vínculos simbólicos es el gasto adicional de procesamiento requerido. Se debe leer el archivo que contiene la ruta, después ésta se debe analizar sintácticamente y seguir, componente por componente, hasta llegar al nodo-i. Toda esta actividad puede requerir una cantidad considerable de accesos adicionales al disco.

Además, se necesita un nodo-i adicional para cada vínculo simbólico, al igual que un bloque de disco adicional para almacenar la ruta, aunque si el nombre de la ruta es corto, el sistema podría almacenarlo en el mismo nodo-i, como un tipo de optimización. Los vínculos simbólicos tienen la ventaja de que se pueden utilizar para vincular archivos en máquinas dondequiera en todo el mundo, con sólo proporcionar la dirección de red de la máquina donde reside el archivo, además de su ruta en esa máquina.

También hay otro problema que introducen los vínculos, ya sean simbólicos o de cualquier otro tipo. Cuando se permiten los vínculos, los archivos pueden tener dos o más rutas. Los programas, que empiezan en un directorio dado, encontrando todos los archivos en ese directorio y en sus subdirectorios, localizarán un archivo vinculado varias veces. Por ejemplo, un programa que vacía todos los archivos en un directorio y sus subdirectorios en una cinta, podría realizar varias copias de un archivo vinculado. Lo que, es más, si la cinta se lee en otra máquina, a menos que el programa de vaciado sea inteligente, el archivo vinculado se copiará dos veces en el disco, en vez de ser vinculado.

Referencias y bibliografía de la sección.

- Tanenbaum 4-3-4.
- Silberschatz 10-5.
- Stallings pág. 563.

El sistema de archivos y su estructura.

Los discos constituyeron el principal tipo de almacenamiento secundario para mantener sistemas de archivos durante años. Tienen dos características que los convirtieron en un medio conveniente para el almacenamiento de múltiples archivos:

1. El disco puede ser reescrito de manera directa; es posible leer un bloque del disco, modificar el bloque y volverlo a escribir en el mismo lugar.
2. Con un disco, se puede acceder directamente a cualquier bloque de información que contenga. Por tanto, resulta simple acceder a cualquier archivo de forma secuencial aleatoria y el conmutar de un archivo a otro sólo requiere mover los cabezales de lectura-escritura y esperar a que el disco termine de rotar.

En lugar de transferir un byte cada vez, las transferencias de E/S entre la memoria y el disco se realizan en unidades de bloques, para mejorar la eficiencia de E/S.

Cada bloque tiene uno o más sectores. Dependiendo de la unidad de disco, los sectores varían entre 32 bytes y 4096 bytes; usualmente su tamaño es de 512 bytes.

Para proporcionar un acceso eficiente y cómodo al disco, el sistema operativo impone uno o más sistemas de archivos, con los que los datos pueden almacenarse, localizarse y extraerse fácilmente.

Un sistema de archivos acarrea dos problemas de diseño bastante diferentes. El primer problema es definir qué aspecto debe tener el sistema de archivos para el usuario. Esta tarea implica definir un archivo y sus atributos, las operaciones permitidas sobre los archivos y la estructura de directorio utilizada para organizar los archivos.

El segundo problema es crear algoritmos y estructuras de datos que permitan mapear el sistema lógico de archivos sobre los dispositivos físicos de almacenamiento secundario.

El propio sistema de archivos está compuesto, generalmente, de muchos niveles diferentes. La estructura que se muestra en la siguiente es un ejemplo de diseño en niveles; cada nivel del diseño utiliza las funciones de los niveles inferiores para crear nuevas funciones que serán, a su vez, utilizadas por los niveles superiores a ese.

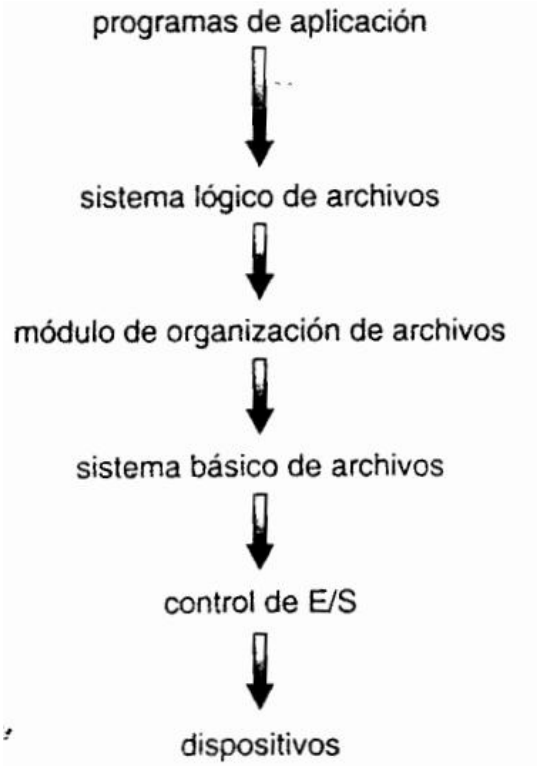


Ilustración 34 - Sistema de archivos en niveles.

El nivel más bajo, el ***control de E/S***, está compuesto por controladores de dispositivo y rutinas de tratamiento de interrupción, que se encargan de transferir la información entre la memoria principal y el sistema de disco.

Un controlador de dispositivo puede considerarse una especie de traductor, su entrada está compuesta por comandos de alto nivel tales como “extraer bloque 123"; su salida consta de instrucciones de bajo nivel específicas del hardware que son utilizadas por la controladora hardware, que es quien establece la interfaz del dispositivo de E/S con el resto del sistema.

El controlador de dispositivo escribe usualmente una serie de patrones de bits específicos en ubicaciones especiales de la memoria de la controladora de E/S, para decir a la controladora en qué ubicación del dispositivo debe operar y que acción debe llevar a cabo.

El ***sistema básico de archivos*** sólo necesita enviar comandos genéricos al controlador de dispositivo apropiado, con el fin de leer y escribir bloques físicos en el disco. Cada bloque físico se identifica mediante su dirección de disco numérica (por ejemplo, unidad 1, cilindro 73, pista 2, sector 10).

El ***módulo de organización de archivos*** tiene conocimiento acerca de los archivos y de sus bloques lógicos, así como de sus bloques físicos. Conociendo el tipo de asignación de archivos

utilizado y la ubicación del archivo, el módulo de organización de archivos puede traducir las direcciones lógicas de bloque a direcciones físicas de bloque, que serán las que envíe al sistema básico de archivos para que realice las necesarias transferencias.

Los bloques lógicos de cada archivo están numerados desde 0 (o 1) a N. Puesto que los bloques físicos que contienen los datos no suelen corresponderse con los números lógicos de bloque, es necesaria una tarea de traducción para localizar cada bloque.

El módulo de organización de archivos incluye también el gestor de espacio libre, que controla los bloques no asignados y proporciona dichos bloques al módulo de organización de archivos cuando así se solicita.

Finalmente, el ***sistema lógico de archivos*** gestiona la información de metadatos. Los metadatos incluyen toda la estructura del sistema de archivos, excepto los propios datos (es decir, el propio contenido de los archivos).

El sistema lógico de archivos gestiona la estructura de directorio para proporcionar al módulo de organización de archivos la información que éste necesita, a partir de un nombre de archivo simbólico. Este nivel mantiene la estructura de los archivos mediante bloques de control de archivo.

Un bloque dé control de archivo (FCB, file-control block) contiene información acerca del archivo, incluyendo su propietario, los permisos y la ubicación del contenido del archivo.

El sistema lógico de archivos también es responsable de las tareas de protección y seguridad.

Cuando se utiliza una estructura de niveles para la implementación de un sistema de archivos, se minimiza la duplicación de código. El código correspondiente al control de E/S y, en ocasiones, al sistema básico de archivos puede ser utilizado por múltiples sistemas de archivos. Cada sistema de archivos puede tener entonces su propio sistema lógico de archivos y su propio módulo de organización de archivos.

Hoy en día se utilizan muchos sistemas de archivos distintos. La mayoría de los sistemas operativos soportan más de un sistema. Por ejemplo, la mayoría de los CD-ROM están escritos en el formato ISO 9660[12], que es un formato estándar adoptado por consenso de los fabricantes de CDROM.

Además de los sistemas de archivos para soportes extraíbles, cada sistema operativo tiene un (o más de un) sistema de archivos basado en disco. UNIX utiliza el sistema de archivos UNIX (UFS, UNIX file system). BSD utiliza FFS (Fast File System) de Berkeley. Windows NT, 2000 y XP soportan los formatos de sistema de archivos de disco FAT, FAT32 y NTFS (Windows NT File System), además de formatos de sistemas de archivos para CD-ROM, DVD y disquete.

Aunque Linux soporta más de cuarenta sistemas de archivos distintos, el sistema de archivos estándar en Linux se denomina sistema de archivos extendido, siendo las versiones más comunes ext2, ext3 y ext4.

También existen sistemas de archivos distribuidos, en los que un sistema de archivos que reside en un servidor puede montarse en uno o más clientes.

[12] Es un sistema de archivos estándar internacional diseñado para discos ópticos (CD-ROM). Se usa para que los mismos discos sean legibles en distintos sistemas operativos.

Referencias y bibliografía de la sección.

- Silberschatz 11-1.

Sistemas de archivos en Linux y Windows.

El sistema de archivos virtual de Linux.

Linux incluye una utilidad versátil y potente para gestión de archivos, diseñada para soportar una gran variedad de sistemas de gestión de archivos y estructuras de archivos. El enfoque usado en Linux consiste en hacer uso del ***sistema de archivos virtual*** (VFS), que presenta una única y uniforme interfaz de sistema de archivos para los procesos de usuario.

VFS define un modelo de archivos común que es capaz de representar cualquier característica general y comportamiento de un sistema de archivos concebible. VFS asume que los archivos son objetos de un sistema de almacenamiento masivo de computadora que comparten propiedades básicas sin tener en cuenta el sistema de archivos concreto o el hardware subyacente.

Los archivos tienen nombres simbólicos que les permiten identificarse de forma única dentro de un directorio específico en el sistema de archivos. Un archivo tiene un propietario, protección frente a accesos o modificaciones no autorizadas y otras propiedades.

Un archivo se puede crear, leer, escribir o borrar. Para cualquier sistema de archivos específico, se necesita un módulo de proyección que transforme las características del sistema de archivos real a las características esperadas por el sistema de archivos virtual.

La siguiente indica los ingredientes clave de la estrategia del sistema de archivos Linux.

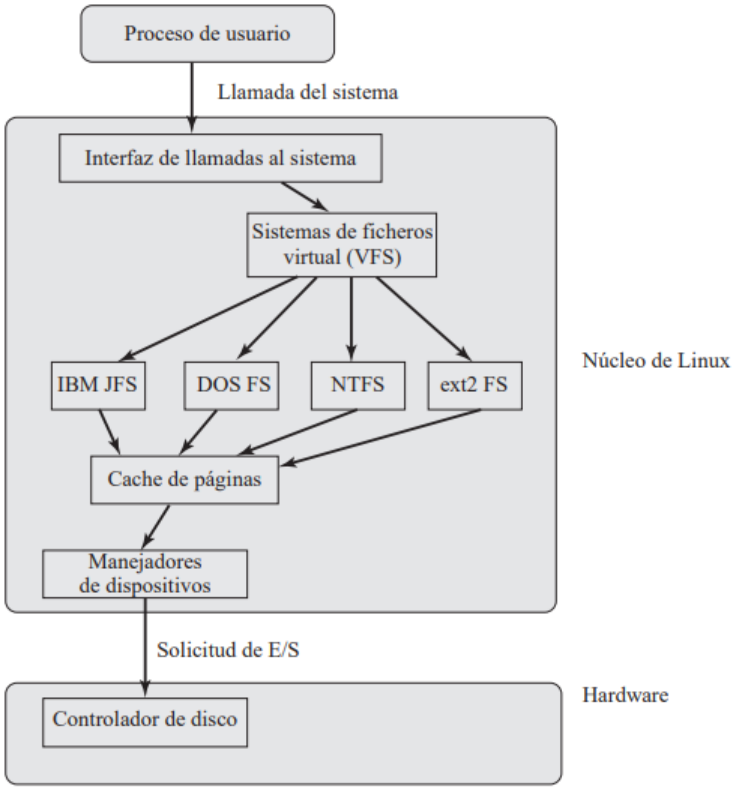


Ilustración 35 - Contexto del sistema de ficheros virtual de Linux.

Un proceso de usuario invoca una llamada al sistema de archivos (por ejemplo, read) utilizando el esquema de archivos VFS. VFS convierte esta llamada en una llamada al sistema de archivos interno (del núcleo) que se pasa a una función de proyección del sistema de archivos específico (por ejemplo, el sistema de archivos JFS[13] - Journaling File System - de IBM).

En la mayoría de los casos, la función de proyección es simplemente una proyección de las llamadas funcionales del sistema de archivos desde un esquema a otro. En algunos casos, la función de proyección es más compleja.

Por ejemplo, algunos sistemas de archivos utilizan una tabla de asignación de archivos (File Allocation Table, FAT), que almacena la posición de cada archivo en el árbol de directorios. En estos sistemas de archivos, los directorios no son archivos. En cualquier caso, la llamada original al sistema de archivos se traduce en una llamada que es nativa para el sistema de archivos destino. El software del sistema de archivos destino es invocado entonces para llevar a cabo la función requerida sobre el archivo o directorio, bajo su control y almacenamiento secundario. Los resultados de la operación se comunican de nuevo al usuario de una forma similar.

[13] Journaling File System (JFS) es un sistema de archivos de 64-bit con respaldo de transacciones creado por IBM.

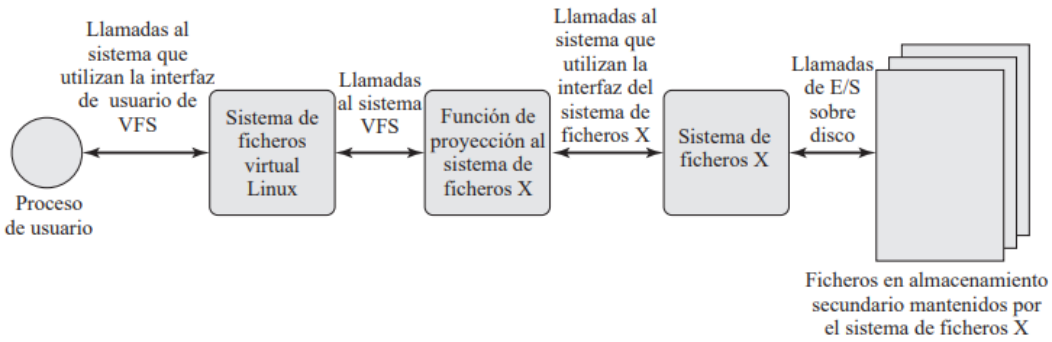


Ilustración 36 - Concepto del sistema de ficheros virtual de Linux.

La figura anterior indica el papel que VFS juega dentro del núcleo de Linux. Cuando un proceso inicia una llamada al sistema orientada a archivos (por ejemplo, lectura), el núcleo llama a una función en VFS. Esta función gestiona los aspectos independientes del sistema de archivos e inicia una llamada a una función en el código del sistema de archivos destino. Esta llamada pasa a través de una función de proyección que convierte la llamada VFS en una llamada al sistema de archivos destino.

VFS es independiente de cualquier sistema de archivos, de modo que la implementación de una función de proyección debe ser parte de la implementación de un sistema de archivos en Linux.

El sistema de archivos destino convierte la petición del sistema de archivos en instrucciones orientadas a dispositivo que se pasan al controlador del dispositivo mediante funciones de cache de páginas.

VFS es un esquema orientado a objetos. Debido a que está escrito en C, en lugar de en un lenguaje que dé soporte a la programación de objetos (como C++ o Java), los objetos VFS se implementan simplemente como estructuras de datos C.

Cada objeto contiene tanto datos como punteros a las funciones implementadas del sistema de archivos que operan sobre los datos.

Los cuatro tipos de objetos primarios en VFS son los siguientes:

- **Objeto superbloque.** Representa un sistema de archivos montado específico.
- **Objeto nodo-i.** Representa un archivo específico.
- **Objeto entrada de directorio.** Representa una entrada de directorio específica.
- **Objeto de archivo.** Representa un archivo abierto asociado con un proceso.

Los conceptos clave del sistema de archivos de UNIX son los siguientes:

Un sistema de archivos está compuesto por una organización jerárquica de directorios.

Un directorio es análogo a una carpeta y puede contener archivos y/u otros directorios.

Debido a que un directorio puede contener otros directorios, se forma una estructura de árbol.

Un camino a través de la estructura de árbol desde la raíz está formado por una secuencia de entradas de directorio, acabando en una entrada de directorio (entradaD) o un nombre de archivo.

En UNIX, un directorio se implementa como un archivo que lista los archivos y directorios contenidos en él. Por tanto, las operaciones de archivos se pueden llevar a cabo tanto sobre archivos como directorios.

El objeto superbloque.

El objeto superbloque almacena información que describe un sistema de archivos específico. Típicamente, el superbloque corresponde al superbloque del sistema de archivos o bloque de control del sistema de archivos, que se almacena en un sector especial en el disco.

El objeto superbloque está formado por varios elementos de datos. Ejemplos de estos elementos incluyen los siguientes:

- El dispositivo sobre el cual el sistema de archivos está montado.
- El tamaño de bloque básico del sistema de archivos.
- El dirty flag, que indica que se ha cambiado el superbloque, pero no se ha escrito a disco.
- Tipo de archivo.
- Flags, como el de sólo lectura.
- Puntero al directorio raíz del sistema de archivos.
- Lista de archivos abiertos.
- Semáforo para controlar el acceso al sistema de archivos.
- Lista de operaciones de superbloque.

El último elemento en la lista precedente se refiere a un objeto de operaciones contenido en el objeto superbloque. El objeto operación define los métodos de objeto (funciones) que el núcleo puede invocar sobre el objeto superbloque.

Los métodos definidos para el objeto superbloque incluyen las siguientes:

- **read_inode.** Leer un nodo-i específico desde un sistema de archivos montado.
- **write_inode.** Escribir un nodo-i dado a disco.
- **put_inode.** Obtener un nodo-i.
- **delete_inode.** Borrar un nodo-i del disco.
- **notify_change.** Invocado cuando se cambian los atributos.
- **put_super.** Llamado por VFS cuando el sistema de archivos está desmontado, para obtener el superbloque dado.
- **write_super.** Invocado cuando VFS decide que el superbloque necesita escribirse en disco.
- **statfs.** Obtiene las estadísticas del sistema de archivos.
- **remount_fs.** Llamado por VFS cuando el sistema de archivos es montado de nuevo con nuevas opciones de montaje.
- **clear_inode.** Obtener nodo-i y borrar cualquier página que contenga datos relacionados.

El objeto nodo-i.

Un ***nodo-i*** está asociado con cada archivo. El objeto nodo-i contiene toda la información sobre un determinado archivo excepto su nombre y el contenido real del archivo.

Los elementos contenidos en un objeto nodo-i incluyen el propietario, grupo, permisos, tiempos de acceso para un archivo, tamaño de los datos que contiene y número de enlaces.

El objeto nodo-i también incluye un objeto operaciones de nodo-i, que describe las funciones implementadas en el sistema de archivos que ***VFS*** puede invocar sobre el nodo-i. Los métodos definidos por el objeto nodo-i incluyen los siguientes:

• **create.** Crear un nodo-i nuevo para un archivo regular asociado con un objeto entradaD en algún directorio.

• **lookup.** Buscar en un directorio el nodo-i correspondiente a un nombre de archivo.

• **mkdir.** Crear un nuevo nodo-i para un directorio asociado a un objeto entradaD en algún directorio.

El objeto entradaD.

Una entradaD (entrada de directorio) es un componente específico de una ruta. El componente puede ser un nombre de directorio o un nombre de archivo. El objeto entradaD facilita el acceso a los archivos y directorios y se utilizan en una cache de entradaD para dicho propósito.

El objeto archivo.

El objeto archivo se utiliza para representar un archivo abierto por un proceso. El objeto se crea en respuesta a la llamada al sistema ***open()*** y se destruye en respuesta a la llamada al sistema ***close().*** El objeto archivo está formado por un conjunto de elementos, que incluye los siguientes:

- Objeto entradaD asociado con el archivo.
- Sistema de archivos que contiene el archivo.
- Contador de uso del objeto archivo.
- Identificador del usuario.
- Identificador de grupo del usuario.
- Puntero de posición del archivo, que es la posición actual en el archivo desde la cual tendrá lugar la siguiente operación.

El objeto archivo también incluye un objeto operaciones de nodo-i, que describe las funciones implementadas del sistema de archivos que VFS puede invocar sobre el objeto archivo. Los métodos definidos para el objeto archivo incluyen la lectura, escritura, apertura, creación y bloqueo.

El sistema de archivos de Windows.

Windows da soporte a varios sistemas de archivos, incluyendo el sistema ***FAT*** (File Allocation Table, tabla de asignación de archivos) que se ejecutaba en Windows 95, MS-DOS y OS/2.

Los desarrolladores de Windows desarrollaron posteriormente otro sistema de archivos, el sistema de archivos de Windows (***NTFS***), que está pensado para alcanzar requisitos de altas prestaciones en estaciones de trabajo y servidores.

Ejemplos de aplicaciones de altas prestaciones incluyen las siguientes:

- Aplicaciones cliente/servidor tales como los servidores de archivos, servidores de computación y servidores de bases de datos.
- Ingeniería intensiva de recursos y aplicaciones científicas.
- Aplicaciones de red para grandes sistemas corporativos.

Características de NTFS.

NTFS es un sistema de archivos flexible y potente, construido, en un modelo de sistema de archivos elegantemente simple. Las características más notables de NTFS incluyen las siguientes:

• **Recuperación.** Uno de los requisitos más importantes del sistema de archivos Windows es la capacidad de recuperarse frente a errores en el sistema y los fallos de disco. En el caso de dichos fallos, NTFS es capaz de reconstruir volúmenes de disco y devolverlos a un estado consistente.

Esto se lleva a cabo utilizando un modelo de procesamiento de transacciones para los cambios en el sistema de archivos; cada cambio significativo se trata como una acción atómica que se realiza de forma completa o no se lleva a cabo en absoluto.

Cada transacción que está en proceso cuando se produce un fallo es a continuación terminada completamente o bien se deja el sistema como antes de su ejecución.

Adicionalmente, NTFS utiliza almacenamiento redundante para datos del sistema de archivos críticos, de forma que un fallo en el sector de un disco no cause la pérdida de datos que describen la estructura y estado del sistema de archivos.

• **Seguridad.** NTFS utiliza el modelo de objetos de Windows para forzar la seguridad. Un archivo abierto se implementa como un objeto archivo con un descriptor de seguridad que define sus atributos de seguridad.

• **Discos y archivos grandes.** NTFS soporta discos y archivos muy grandes de forma más eficiente que buena parte del resto de los sistemas de archivos, incluyendo FAT.

• **Múltiples flujos de datos.** Los contenidos reales de un archivo se tratan como un flujo de bytes. En NTFS es posible definir múltiples flujos de datos para un único archivo.

• **Facilidad general de indexación.** NTFS asocia una colección de atributos con cada archivo. El conjunto de descripciones de archivo en el sistema de gestión de archivos se organiza como una base de datos relacional, de forma que los archivos se pueden indexar por cualquier atributo.

Referencias y bibliografía de la sección.

- Stallings pág. 578.